Intrinsic Memory in Digital SNNs: Enhancing Performance on the Cart-Pole Task with Fractional-Order Dynamics

by

Niklas Anderson

A thesis submitted in partial fulfillment of the
requirements for the degree of

Master of Science
in
Electrical and Computer Engineering

Thesis Committee:
Christof Teuscher, Chair
Yuchen Huang
Xiaoyu Song

Portland State University
2026

Abstract

Researchers working with Spiking Neural Networks (SNNs) are faced with the challenge of identifying useful biologically plausible features for inclusion in neuron models. Prior research indicates that dynamics of fractional-order calculus, which impart a form of intrinsic memory, are a key feature of biological neurons. In this work, we present the development and usage of neural networks with intrinsic memory in application to the cart-pole task, a common benchmark for neural network performance. We found that usage of fractional-order neurons may allow for up to a 37.5% reduction in neural network size as compared to networks using a standard Leaky Integrate-and-Fire (LIF) neuron. In addition, fractional-order dynamics appear to be a particularly effective way to incorporate intrinsic memory, as we demonstrate that a method leveraging bitshifts instead of multiplication for inclusion of history required a higher number of neurons for learning the cart-pole task. Though we report challenges in obtaining consistent training results using a Deep Q-Networks (DQN) algorithm, our analysis of over 100 trials for each of three neural network types supports our conclusions regarding trend in network sizes across models. The primary tradeoffs we identify for neurons with intrinsic memory are the additional resources to support calculation of a history term. We report these added requirements in the context of implementation and execution in a digital computing environment. Through our investigation, we created a software-to-hardware workflow for testing novel neuron models, beginning with training a network in software and resulting in execution on hardware. This work supports future research into custom neuron models, providing a pathway for

evaluation of features, such as fractional-order dynamics, which may prove useful in improving accuracy and reducing network size. We have identified that fractional-order dynamics specifically show promise in enhancement of neural network performance, and this work may inspire future development of analog components with inherent fractional-order capabilities.

Acknowledgements

There are many people who have played a role in helping me complete my MS degree and this thesis. First of all, I want to thank my advisor, Christof Teuscher, for providing me with opportunities to expand my knowledge and exposure to the academic world of unconventional and next-generation computing. He has been a consistent source of practical advice, always with a push to refine, revise, and polish my work. I want to thank my committee members, Yuchen Huang and Xiaoyu Song. In addition to serving on this committee, Professor Huang has led me through the advanced computer architecture series with intelligence, kindness, and an unwavering commitment to helping students succeed. Professor Song has provided insights into synthesizeable SystemVerilog and creative analogies for explaining concepts such as setup and hold time. I will continue to apply the techniques and concepts they have taught me throughout my career.

Thank you to Reece Wayt and Truong Le, who have been consistently great teammates as we've made our way through the ECE MS program. I admire the dedication they both have to producing high-quality work. And a thank you to Tucker Mastin, my fellow Teuscher Lab member, for helping me understand underlying concepts and sharing his enthusiasm for this material.

Support from my family has kept me going through this work over the last year, and beyond. I want to thank Lindsay Winsor, who has asked thoughtful questions, and helped me see big picture goals. To my parents, Kirsten Erickson and Patrick Anderson, I want to express my gratitude for their steadfast support over a lifetime.

Their encouragement of my goals and academic pursuits have kept me going over the long haul.

Last but never least, I want to thank my partner, Jill, who has always made my world larger, even while I've spent months diving deep into the single-minded task of thesis completion. She's supported me at the lowest moments, and celebrated with me at the highest. I simply could not have done this without her.

Table of Contents

List of Tables

List of Figures

List of Listings

List of Acronyms

ADM Adomian Decomposition Method. 8

AI Artificial Intelligence. 1, 15

ANN Artificial Neural Network. 1, 7, 14, 49

BRAM Block Random Access Memory. ix, 83

CORDIC COordinate Rotation DIgital Computer. 9

CPU Central Processing Unit. 57

DQN Deep Q-Networks. i, 14–16, 49, 66, 67, 93, 95

DSP Digital Signal Processing. viii, ix, xv, 3, 26, 31, 43, 45, 47, 70, 77–80, 83–85, 89–91, 94

FPGA Field Programmable Gate Array. v, ix, xiv, 3–11, 26, 37, 39, 43, 44, 49, 54, 59, 70–73, 75, 76, 78, 86, 87, 92, 94

FSM Finite State Machine. xv, xvi, 37, 71, 78, 85, 87, 91, 92, 94

GL Grünwald-Letnikov. 4, 8, 17, 18, 25, 26, 31, 33, 34, 45, 93

GPU Graphics Processing Unit. 1, 57, 67

HDL Hardware Description Language. vi, xiii, 2–6, 10, 16, 25, 26, 30, 35, 53–55, 58, 68, 93, 95

HR Hindmarsh Rose. 9, 45

IF Integrate-and-Fire. 11

IQM Inter-Quartile Mean. xiii, xiv, 56, 58–63, 65

IQR Inter-Quartile Range. 63

ISI Inter-Spike Interval. xi, 40–42

LED Light-Emitting Diode. xiv, 74

LIF Leaky Integrate-and-Fire. i, vi, viii, ix, xi–xvi, 5, 7, 10, 11, 17, 25, 26, 30, 31, 35–37, 39, 40, 43–45, 49–52, 59–65, 67–70, 78, 79, 82–94

LLM Large Language Model. 1

LUT Look-Up Table. ix, 77, 78, 83–86, 89, 91, 94

MAC Multiply-Accumulate. xv, 38, 39, 43, 80

PPO Proximal Policy Optimization. 14, 49, 67, 95

RC Resistor-Capacitor. 16

RL Reinforcement Learning. 13, 15, 16, 49, 50, 56

SIMT Single Instruction, Multiple Thread. 68

SNN Spiking Neural Network. i, v, ix, 1–3, 5–7, 9, 10, 13, 14, 48–50, 65, 66, 68, 70, 75, 87, 93–95

SPI Serial Peripheral Interface. 72

STDP Spike-Timing Dependent Plasticity. 11

STDP-RL Spike-Timing-Dependent Reinforcement Learning. 14

SV SystemVerilog. vii, xii, xvii, 30, 35, 49, 50, 52–54, 64, 74, 76, 78, 90, 95

UART Universal Asynchronous Receiver-Transmitter. xiv, 70–75

USB Universal Serial Bus. 72

WNS Worst Negative Slack. viii, 45

Chapter 1

Introduction

1.1 Problem Definition

In a report released in 2025 on Artificial Intelligence (AI) and energy usage, the International Energy Agency estimated a typical AI-focused data center consumed as much electricity as 100,000 households, and the largest ones under construction at the time of the report would consume 20 times as much [1]. These estimates largely assume the continued use of current AI technologies, such as Artificial Neural Networks (ANNs) running on Graphics Processing Units (GPUs), though the report authors provide insight into an alternative scenario where more efficient approaches to building AI are used. Neuromorphic computing has the potential to make a major contribution to a more energy-efficient future for AI. In a recent implementation of a Large Language Model (LLM) adapted for execution on neuromorphic hardware, Abreu et al. [2] reported energy usage as little as half of the standard approach using a GPU. Beyond reducing the overall energy costs of AI, neuromorphic computing may also serve as a key technology in deploying AI solutions in resource-constrained edge computation contexts.

Though results reported in [2] and [3] provide credible demonstrations that neuromorphic computing and its SNNs offer promising improvements over ANNs, especially with regard to energy efficiency, the on-ramp for AI model developers to leverage these alternatives is not always straightforward. Developers working to produce SNN models which may be converted for execution in custom hardware face an uncertain

pathway in doing so. These challenges are compounded for researchers who wish to explore unusual or novel neuron models.

At the same time, exploration of biologically plausible characteristics of neurons may be necessary to unlock improvements in efficiency and accuracy. There are many possible biologically plausible features, and narrowing focus to the most meaningful or impactful of these is important in determining how to realize those improvements in digital hardware.

Theoretical research has identified fractional-order dynamics, a feature shown to be a key characteristic of biological neurons [4, 5], as one such characteristic which may be worth further investigation. These dynamics impart the characteristic of intrinsic memory, such that neurons incorporate their membrane history in a way that may improve accuracy at smaller network sizes. Neurons with intrinsic memory may be more biologically plausible, and more importantly, may confer performance advantages in the context of resource-constrained environments. While there are analog capacitor components that are capable of representing fractional-order calculus, these components are not currently in mainstream production [6, 7]. Digital fractional-order systems can be developed on a nearer-term timeline, and could be used to inform the process and provide motivation for development of analog fractional-order components.

1.2 Research Goals and Significance

In this thesis, we address intrinsic memory in SNNs and make explicit the hardware requirements for representing this biological feature in a digital implementation. We present an engineering-focused, methodical approach to the assessment of design space options and considerations for the modeling of neurons with intrinsic memory in digital systems using HDL. Based on the results of design space analysis, simulation, and synthesis, a digital fractional-order neuron does appear to be a viable option for future

work involving the creation of neural networks with intrinsic memory. Our work may provide guidance for future research that relies on appropriate selection of features such as the fractional-order α value, FPGA board resources such as memory and DSP slices, as well as target neural network and history length size.

Through this process, we have created a software-to-hardware pipeline, which allows for training SNN models using common software tools such as the snnTorch [8] library and Python code, validating models which have been quantized for efficient use with hardware, and simulating models in HDL using the testing tool cocotb [9] and reinforcement learning library gymnasium [10]. This model may be followed by other researchers hoping to incorporate testing of custom neuron models using standard simulation tools. This may allow for more rapid evaluation of biologically plausible features and alternate neuron update mechanics.

Additionally, by building SNN models with intrinsic memory and testing on a standard benchmark task, this thesis aims to provide an in-depth understanding of the strengths and limitations of such models in their execution on digital hardware. Through experimentation with varying means of incorporating neuronal membrane history, we intend to show that fractional-order modeling does indeed make a meaningful difference in neural network performance. This has been done through development of high-performing models with intrinsic memory, and subsequent quantitative analysis of their power and resource usage on an FPGA. This information will inform which avenues are worth pursuing for implementation of such history-dependent systems, and whether there are specific workloads or applications which may be sufficiently addressed by these models.

Finally, we have demonstrated execution on FPGA of a model trained in software with a custom fractional-order neuron. With up to 37.5% reduced neural network size, this model achieves accurate performance on the cart-pole application, a real-time task which has been identified as a standard benchmark for neuromorphic computing

[11]. For applications where minimization of neural network size is critical, our work provides guidance on how to achieve such reductions. Extending the significance beyond the digital realm, providing an FPGA resource usage profile for varying models may also provide motivation for future development of analog components with inherent fractional-order dynamics.

1.3 Contributions

Specific contributions made in this paper to the project of advancing neuromorphic computing and investigation into critical biologically plausible features of neurons include the following:

- The identification of primary design considerations necessary for the implementation of neurons with intrinsic memory in HDL, such as bit widths for values including the fractional-order term α and binomial coefficients associated with the Grünwald-Letnikov (GL) approximation. This work is presented in Chapter 3.

- Digital-implementation sections of *Fractional-order systems for neuromorphic computing: Software and hardware opportunities and challenges* by Mastin et al. [12], based on the work in Chapter 3.

- Exploration of hardware-efficient means to incorporate intrinsic memory into neuron design as an alternative to fractional-order calculations. We present and assess four variants of neurons models with intrinsic memory using bitshift operations, and show that a model using a custom slow decay for neuron membrane potential history terms may provide the closest approximation to fractional-order binomial coefficients for $\alpha = 0.5$. Design of these variants is described in Chapter 3, while analysis of performance is primarily contained in Chapters 4 and 5.

- Design and description of a software-to-hardware workflow which may be used to train SNNs with custom neurons in software for HDL simulation and validation. Our results indicate that a fractional-order SNN may provide a 37.5% reduction from the LIF in neural network size with comparable performance on the cart-pole task, while a bitshift model resulted in a larger network size. This work is presented in Chapter 4.

- Quantification of resources required for digital implementation of two possible means for incorporation of intrinsic memory in SNNs on FPGA, as compared to a baseline SNN using LIF neurons. Results show that true fractional-order dynamics result in 31% reduced latency per cart-pole inference step, while a bitshift method of approximating incorporation of intrinsic memory resulted in no improvements over the LIF, using SNN models with comparable performance on a benchmark task. Due to the additional hardware resources required for history term calculation and inclusion, our fractional-order and LIF networks had comparable total energy usage for inference on the cart-pole task. Chapter 5 contains this work on FPGA execution.

These contributions provide insight into the feasibility of real-world usage of digital SNNs with intrinsic memory and may inform future work related to history-dependent neurons used in both digital and analog systems.

1.4 Document Overview

The following chapters provide a review of the literature, relevant background, and detailed descriptions of the methods and results used at each stage of our work. We discuss our findings and conclusions, as well as provide ideas for future research. In chapter 2, we review related work and current best practice, and provide background on relevant concepts. Chapter 3 covers design space analysis for digital implementation

of neurons with intrinsic memory, including fractional-order as well as alternative methods for creating history dependence in neurons. In Chapter 4, we detail the process of training SNNs in software on the cart-pole task benchmark, validating a quantized model, and performing simulation of these models in HDL. In Chapter 5, we show the results of synthesizing our HDL modules and executing optimized models on an FPGA. In Chapter 6, we summarize our contributions and describe limitations and future work.

Chapter 2

Literature Review and Background

2.1 Literature Review

The following literature review covers the current usage and challenges facing neuromorphic computing, applications of fractional-order calculus in digital systems, and implementations targeting FPGAs specifically. We also address training and benchmarking for neuromorphic systems.

2.1.1 SNNs and Neuron Models

Neuromorphic computing and its SNNs have the potential to surpass ANNs, particularly in terms of power consumption, as Kannan et al. [13] highlight the application of neuromorphic computing for ultra-low power consumption, and Yamazaki et al. [14] note the overall energy efficiency and related sparsity of SNNs. However, challenges for widespread adoption of SNNs relate to a lack of efficient training algorithms as described in [15] and a more general need for standardized toolchains detailed in [3]. And while Yamazaki et al. [14] and Tang et al. [15] emphasize the importance of exploring biologically plausible features of neurons in the context of SNN development, most existing neuromorphic hardware and software does not readily accommodate custom neuron models.

Teka et al. [4, 16] have identified fractional-order dynamics as an especially consequential feature in biological neurons, which results in a neuron's ability to maintain a form of intrinsic memory. The more commonly adopted LIF neuron is much simplified

from more complex biologically-inspired models [14, 15], and does not contain any similar mechanism for incorporating neuron membrane history. Despite this gap between typical implementation and the theory, analog components that inherently show fractional-order behavior are still in development and not ready for widespread use [6, 17].

2.1.2 Digital Fractional-Order Systems

Regarding a general approach to implementation of fractional-order calculus in digital systems, both review articles [6] and [18] contain mathematical grounding through an overview of fractional-order definitions and options for numerical approximations. Clemente-López et al. [6] address fractional-order systems on both FPGAs and embedded hardware, while Ali et al. [18] focus on FPGAs and their unique architectural features. Both perform tradeoff analysis between different numerical approximation methods, and discuss topics most relevant to digital systems, such as discretization techniques and usage of floating point versus fixed-point representations. Clemente-López et al. [6] highlight the Adomian Decomposition Method (ADM) as well as the aforementioned GL method as the two primary approximation methods used in digital implementations. The authors further note that the GL method is generally preferred due to its simplicity and flexibility. In [18], Ali et al. additionally describe usage of the popular Tustin method for discretization.

The above review articles are in general agreement that the GL method for fractional-order approximation is well suited for digital platforms. In addition, they both cite the choice to use fixed-point as an appropriate design decision for numerical representation. While both articles include details on implementation of digital fractional-order systems, only a small subset of the reviewed work targets neuron models or applications for neural networks.

2.1.3 Neurons and SNNs on FPGAs

Most existing work on implementing SNNs for digital contexts covers realization of common neuron models and neural networks on FPGA. Thomas and Luk [19] offer a biologically plausible SNN targeting FPGA execution using the Izhikevich neuron model, showing at a baseline that FPGAs are a reasonable environment for execution of moderately complex models with a speedup over standard CPU execution. Shama et al. [20] and Ghanbarpour et al. [21] further show that increasingly complex neuron models may be represented on FPGA, while maintaining a balance in hardware complexity. Targeting even more specific optimizations, Heidarpour et al. [22] propose usage of a COordinate Rotation DIgital Computer (CORDIC) model to reduce digital hardware costs. Though these articles do not address fractional-order computations directly, they may serve to provide models for efficient digital implementations of complex neuron models.

Fractional-Order Neurons In [23], Malik and Mir present a Hindmarsh Rose (HR) neuron with fractional-order update dynamics. Their work spans single neuron behavior as well as the combined behavior resulting from a coupling of two neurons, with execution of neuron activity on an FPGA. The authors include waveforms showing neuronal dynamics and a range of spiking behavior. Details on representation in hardware include FPGA resource utilization and notes describing the use of a 32-bit fixed-point format. However, an in-depth description of the hardware implementation and tradeoff analysis is not provided.

Similarly, Tolba et al. [24] outline an approach for fractional-order update dynamics in an Izhikevich neuron model executing on an FPGA. They also produced waveforms showing the resulting spiking behaviors, as well as block-level hardware diagrams. Though their work includes greater depth regarding details such as bit widths and routing between high-level components, the focus of the article tends towards the

comparison between the integer-order and fractional-order implementations, such that that the information on engineering tradeoffs and their implications for the fractional-order alone is limited.

Taken as a whole, the set of related work offers insight into what decisions are within the scope of the design space for digital implementation of fractional-order neuron models. While no single article contains a comprehensive review of the factors required for consideration in implementation of a fractional-order neuron model such as an LIF in HDL, their individual contributions may be consolidated into a high-level examination of the design space.

SNN Frameworks for FPGAs An increasing number of researchers have begun to focus their efforts on creating SNN frameworks for FPGAs. These frameworks vary in their placement on a flexibility to ease-of-use spectrum, with efforts such as NeuroCoreX [25] and FPGA-NHAP [26] allowing for a high degree of user customization, while Caspian [27] provides an opinionated approach geared towards quickly onboarding users who may have minimal familiarity with neuroscience. Target execution context is another area where current solutions diverge, with frameworks such as Caspian [27] and Spiker+ [28] positioning themselves as compatible with edge deployments in resource-constrained environments. In contrast, FireFly v2 [29] promotes high throughput and performance at the cost of energy-efficiency. Though we were unable to find any SNN frameworks for FPGAs that allowed for usage of fractional-order neurons, these frameworks are useful in that they provide insights into a range of performance characteristics, features, and topologies used when implementing full neural networks on FPGA rather than standalone neurons such as those reviewed in Section 2.1.3.

Tables 2.1 and 2.2 group these frameworks at a high level as having either a focus on learning and research contexts (Table 2.1) versus performance (Table 2.2), though

it should be noted that the cross-section of features from each is quite heterogeneous. These solutions cover a variety of behaviors on attributes such as connectivity type, hardware techniques used, and maximum number of neurons and synapses on a given FPGA board. However, there is convergence on other attributes, leaving room for extension of these existing tools or creation of new frameworks. One such example is in the supported neuron models, where FPGA-NHAP [26] is the only option to provide an Izhikevich model, with other frameworks focusing their efforts on variants of Integrate-and-Fire (IF) or LIF. Similarly, NeuroCoreX [25] is the only framework reviewed which has an on-chip learning mechanism, through their variant of Spike-Timing Dependent Plasticity (STDP). The remaining frameworks require off-chip training and conversion of weights to match the platform-specified format.

Table 2.1: Learning and Adaptive SNN FPGA Frameworks

Feature	**Spiker+ [28]**	**NeuroCoreX [25]**	**Caspian [27]**
Primary Goal	Efficient edge inference	Flexible emulation and on-chip learning	Rapid prototyping and SWaP optimization
Main Differentiator	Python-to-VHDL generation	On-chip STDP; support for non-layered networks	Simplified IP core design
Supported Neuron Models	IF, I-Order LIF, II-Order LIF	LIF (current-based)	LIF (integer-based)
Connectivity Type	Feedforward, fully connected, recurrent	All-to-all (reconfigurable)	Arbitrary directed graph
Training Method	Offline (BPTT, surrogate gradients)	Online STDP / offline	Offline (SLAYER, EONS)
Core Hardware Technique	Parallel layer update	Time-multiplexed single neuron	Small IP core (μCaspian)
Limiting Resource	BRAM	BRAM (Artix-7)	LUTs/BRAM (iCE40 UP5k)
Weight Representation	16-bit fixed-point (two's complement)	8-bit signed	8-bit signed
Max Neurons	$\sim$1,220 (XC7Z020)	100 (scalable)	256
Max Synapses	Not specified	10,000 (+10,000 inputs)	4,096
FPGA Used	XC7Z020	Artix-7	iCE40 UP5k
Clock Speed	100 MHz	100 kHz (neural processing)	Not specified
Power Usage	180 mW (MNIST)	305 mW	Low milliwatt range
Key Benchmark	MNIST: 93.85% (780 μs/img)	DIGITS: 68% (fidelity-focused)	NMNIST: 98.28% (simulation)

Table 2.2: Architecture and Performance-Focused SNN FPGA Frameworks

Feature	**FireFly v2 [29]**	**FPGA-NHAP [26]**
Primary Goal	High-performance algorithm support	General scalable neuron platform
Main Differentiator	DSP-based non-spiking execution	RISC-V-based configuration architecture
Supported Neuron Models	IF, LIF, residual membrane potential	LIF, Izhikevich
Connectivity Type	Convolutional, residual connections	Fully connected
Training Method	Offline (BrainCog)	ANN-SNN conversion
Core Hardware Technique	Systolic array, spatiotemporal dataflow	16-parallel neuron time-multiplexing
Limiting Resource	DSP + routing	Off-chip memory access
Weight Representation	8-bit fixed-point	16-bit fixed-point
Max Neurons	Not architecture-limited	16,384
Max Synapses	Not architecture-limited	16.8M
FPGA Used	ZCU104 (XCZU5EV)	Kintex-7 XC7K325T
Clock Speed	500–600 MHz	200 MHz
Power Usage	4.9 W	535 mW
Key Benchmark	MNIST: 98.2%	MNIST: 97.7%

2.1.4 The Cart-Pole Benchmark Task

A potential application for SNNs is on control tasks in the sensory-motor domain which rely on Reinforcement Learning (RL). One representative exercise in this area is the cart-pole task, popularized as a neuromorphic benchmark by Barto, et al. [30]. In this task, an agent receives inputs from a cart with an inverted pendulum moving on a track and outputs a directional signal in the goal of keeping the pendulum, or pole, upright for as many timesteps as possible. Inputs include the cart position on the track, cart velocity, angle of the pole with the vertical, and the pole angle rate of

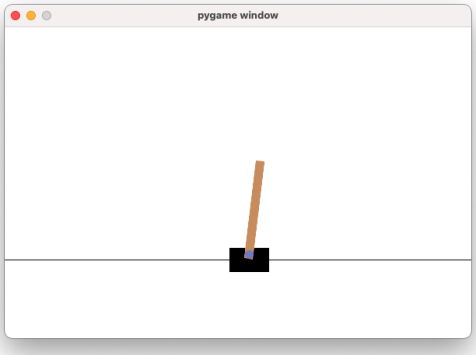

Figure 2.1: Cart-pole task rendering from the Python library Gymnasium [31]. The four values representing the cart's state are the cart position on the track, cart velocity, angle of the pole with the vertical, and the pole angle rate of change.

change. These four values represent the state of the cart. Figure 2.1 shows the task as rendered by the Python library Gymnasium [31].

Training SNNs for benchmarking on the cart-pole trial has included approaches such as using Proximal Policy Optimization (PPO) [32], and Spike-Timing-Dependent Reinforcement Learning (STDP-RL) or evolutionary strategies [33]. Additionally, an introductory tutorial in [34] outlines several standardized approaches which may be used by ANNs as well as SNNs, including DQN, Double DQN, dueling networks, and prioritized experience replay.

Efforts to standardize approaches to the cart-pole trial such that it may serve as a common benchmark are presented by Plank et al. [11], where they include several tiers of difficulty and lay out constraints within which researchers should test their models. One such metric that currently lacks standardization is the maximum number of timesteps in an episode, which determines the total maximum reward. As noted in [11], the default number of timesteps in the current version of the Gymnasium cart-pole environment is 500 [10], while Plank et al. recommend 15,000. Maximum timesteps in [34] is 200, while both [32] and [33] use 500.

Further, of particular note when considering development of fractional-order SNNs,

Plank et al. describe the cart-pole task as a RL task in which it may be beneficial for the agent to "maintain some system state in its 'memory' " [11, p. 3]. Fractional-order systems may be uniquely well-positioned to maintain such system state in tasks like the cart-pole benchmark.

2.1.5 Deep-Q Learning

RL is a means to train AI models using direct feedback from a given environment, where an agent is tasked with optimizing rewards which are returned based on the action taken in a given state. Q-learning is an RL algorithm where a basic lookup table with action rewards is updated over the course of training. Mnih et al. [35] provide an extension of the Q-learning algorithm that may be used to train neural networks, called Deep Q-learning, which has better scalability than Q-learning. The result of Deep Q-learning is a DQN, a neural network trained to maximize rewards in an environment by taking actions based on weights developed during a training period.

The training approach from Mnih et al. [35] involves a replay mechanism, where prior transitions are randomly sampled in a manner that reduces the impact of sequential states and stabilizes training. The replay memory also increases sample efficiency by reusing transitions multiple times during training. DQN uses a target network in addition to a policy network, where the policy network is used for determining the next action in the environment and the predicted Q-value when training on the replay buffer data; the target network is used as a stable means to produce the bootstrapped Q-target, as it is treated as a constant and not subject to the gradient update. While the policy network is updated directly through gradient updates using the loss calculation, the target network is updated periodically from the policy network. Though Mnih et al. [35] utilized a hard update where the network is copied after a set number of steps, Lillicrap et al. [36] subsequently proposed a soft update where the

target network is pushed slightly towards the policy network at every step, with the rate determined by the hyperparameter τ. Their goal in proposing the soft update is to enhance stability by frequent, small updates to the target network. The DQN model with soft target network update may be applied to control problems such as the cart-pole task, as seen in the PyTorch RL tutorial [37].

2.2 Background

Background information is presented here related to the mathematical foundations for the neuron dynamics used in the following implementations in HDL. We begin with review of a standard neuron membrane potential update equation. We then show derivation of a fractional-order model, and finally we provide a small example calculation using a fractional-order approximation.

2.2.1 Membrane Potential Update Equation

Authors in [38] provide a basic membrane potential update equation based on Resistor-Capacitor (RC) circuit dynamics. This serves as the basis for neuron models such as the `Leaky` and `Lapicque` classes in the snnTorch Python library [8]. An equivalent update equation is seen in Equation 2.1, which models the change in membrane voltage over continuous time in terms of the voltage at time t, membrane resistance R, the RC time constant τ, and the input current (I) at time t:

$$\tau \frac{dV(t)}{dt} = -V(t) + RI(t). \tag{2.1}$$

When simplified to assume $R = 1$, application of Euler discretization results in

$$\frac{V[t+1] - V[t]}{\Delta t} = \frac{-V(t)}{\tau} + \frac{I(t)}{\tau}. \tag{2.2}$$

We may then define a decay factor, β, which is equal to $1 - \frac{\Delta t}{\tau}$ and derive

$$V[t+1] = \beta \times V(t) + \frac{I(t) \times \Delta t}{\tau}. \tag{2.3}$$

In treating the final term as a normalized input scale, we can rewrite Equation 2.3 as

$$V[t+1] = \beta \times V(t) + I(t). \tag{2.4}$$

And finally, when adding a membrane potential reset term, where the potential is reset by a specified value after spiking, we achieve an equation matching that found in the actual Python code for the `Leaky` neuron class in [8]:

$$V[t+1] = \beta \times V(t) + I(t) - \text{Reset} \times V_{threshold}, \tag{2.5}$$

where "Reset" is a Boolean value indicating whether or not to include a reset based on spiking behavior in the prior timestep, and $V_{threshold}$ is the voltage threshold for determining whether or not to output a spike.

2.2.2 Fractional-Order Update Equation for LIF Neuron

Fractional-order calculus provides a model for taking non-integer-order, or fractional-order derivatives, using α as the fractional-order term. It serves as a non-Markovian model which includes the entire history of a given value, such as a neuron's membrane potential, or a set history length in the case of a discretized approximation.

The continuous-time fractional-order equation may be defined as

$$D^{\alpha}V(t) = -\lambda \times V(t) + I(t), \tag{2.6}$$

where λ represents a membrane leakage coefficient analogous to $\frac{1}{\tau}$ in the standard LIF Equation 2.1.

As described in section 2.1.2, Clemente-López et al. [6] found that the GL approx-

imation technique for fractional-order representation is one of the most commonly used methods in digital applications. This is likely due to its inherent discretization based on history length and time step, represented in Equation 2.7 as H and Δt, respectively. The GL approximation may be written as

$$D^{\alpha}V(t) \approx \frac{1}{\Delta t^{\alpha}} \sum_{k=0}^{H-1} (-1)^{k} \binom{\alpha}{k} V(t - k\Delta t), \tag{2.7}$$

where $\binom{\alpha}{k}$ are generalized binomial coefficients which essentially serve as weights for historical membrane potential values.

Importantly, in Equation 2.7, the $V(t)$ value at the current timestep is included along with prior values up to $H - 1$. This means it may be simplified to

$$D^{\alpha}V(t) \approx \frac{1}{\Delta t^{\alpha}} [V(t) + \sum_{k=1}^{H-1} (-1)^{k} \binom{\alpha}{k} V(t - k\Delta t)], \tag{2.8}$$

as the binomial coefficient in the 0th position, where $k = 0$, is equal to 1.

The complete discretized equation is found by replacing $D^{\alpha}V(t)$ with the GL

approximation from Equation 2.8 and simplifying to solve for $V(t)$:

$$\frac{1}{\Delta t^{\alpha}}[V(t) + \sum_{k=1}^{H-1}(-1)^k \binom{\alpha}{k} V(t - k\Delta t)] = -\lambda \times V(t) + I(t)$$

$$\frac{V(t)}{\Delta t^{\alpha}} + \frac{1}{\Delta t^{\alpha}} \times \sum_{k=1}^{H-1}(-1)^k \binom{\alpha}{k} V(t - k\Delta t) = -\lambda \times V(t) + I(t)$$

$$\frac{V(t)}{\Delta t^{\alpha}} + \lambda \times V(t) = I(t) - \frac{1}{\Delta t^{\alpha}} \times \sum_{k=1}^{H-1}(-1)^k \binom{\alpha}{k} V(t - k\Delta t)$$

$$V(t) \times [\frac{1}{\Delta t^{\alpha}} + \lambda] = I(t) - \frac{1}{\Delta t^{\alpha}} \times \sum_{k=1}^{H-1}(-1)^k \binom{\alpha}{k} V(t - k\Delta t)$$

$$V(t) = \frac{I(t) - \frac{1}{\Delta t^{\alpha}} \times \sum_{k=1}^{H-1}(-1)^k \binom{\alpha}{k} V(t - k\Delta t)}{[\frac{1}{\Delta t^{\alpha}} + \lambda]}$$

We may treat $\frac{1}{\Delta t^{\alpha}}$ as a constant fractional timestep scaling factor, C, and label the sum $\sum_{k=1}^{H-1}(-1)^k \binom{\alpha}{k} V(t - k\Delta t)$ as the history_term. We can then rewrite in the simplified form

$$V(t) = \frac{I[t] - C \times \text{history\_term}}{C + \lambda}. \tag{2.9}$$

2.2.3 Example Application of Fractional-Order Update Equation

The following shows an example of application of the derived formula presented in Section 2.2.2:

$$V(t) = \frac{I(t) - \frac{1}{\Delta t^{\alpha}} \sum_{k=1}^{H-1}(-1)^k \binom{\alpha}{k} V(t - k\Delta t)}{\left[\frac{1}{\Delta t^{\alpha}} + \lambda\right]}. \tag{2.10}$$

Given Parameters

$$\alpha = 0.5$$

$$\lambda = 0.1$$

$$\Delta t = 1$$

$$H = 4$$

Assumed Values

Input current:

$$I(t) = 1.0$$

Historical membrane potentials (rising values over time):

$$V(t-1) = 0.8$$

$$V(t-2) = 0.5$$

$$V(t-3) = 0.2$$

Step 1: Binomial Coefficients

$$\binom{0.5}{k} = \frac{0.5(0.5-1)\cdots(0.5-k+1)}{k!}$$

$$\binom{0.5}{1} = 0.5$$

$$\binom{0.5}{2} = \frac{0.5(-0.5)}{2} = -0.125$$

$$\binom{0.5}{3} = \frac{0.5(-0.5)(-1.5)}{6} = 0.0625$$

Step 2: Compute the Summation

$$\begin{aligned}\sum_{k=1}^{3}(-1)^k\binom{0.5}{k}V(t-k) = (-1)^1(0.5)(0.8) \\ + (-1)^2(-0.125)(0.5) \\ + (-1)^3(0.0625)(0.2)\end{aligned}$$

$$\begin{aligned}&= (-0.4) + (-0.0625) + (-0.0125) \\ &= -0.475\end{aligned}$$

Step 3: Plug Into Equation

Since $\Delta t^\alpha = 1^{0.5} = 1$:

$$\begin{aligned}V(t) &= \frac{1.0 - (-0.475)}{1 + 0.1} \\ &= \frac{1.475}{1.1} \\ &\approx 1.341\end{aligned}$$

2.2.4 History Term Signedness

The ultimate signedness of the history term has an impact on the types of operations performed and the format used for data such as the binomial coefficients. The following shows the derived usage of a positive history term which is added to the input current instead of using subtraction of a negative term.

We begin again with Equation 2.10, the fractional-order membrane update equation:

$$V(t) = \frac{I(t) - \frac{1}{\Delta t^{\alpha}} \sum_{k=1}^{H-1} (-1)^k \binom{\alpha}{k} V(t - k\Delta t)}{\left[\frac{1}{\Delta t^{\alpha}} + \lambda\right]}. \tag{2.11}$$

As we've defined above, α is fractional:

$$0 < \alpha < 1$$

We now examine the calculation of the history terms and their signedness.

First History Term ($k = 1$)

$$(-1)^1 \binom{\alpha}{1} = -\alpha$$

Since $\alpha > 0$, this coefficient is negative.

Second History Term ($k = 2$)

$$(-1)^2 \binom{\alpha}{2} = (+1)\frac{\alpha(\alpha - 1)}{2}$$

Because:

$$\alpha > 0 \qquad \text{and} \qquad \alpha - 1 < 0$$

the numerator is negative:

$$\alpha(\alpha - 1) < 0$$

Therefore:

$$(-1)^2 \binom{\alpha}{2} < 0$$

Third History Term ($k = 3$)

$$(-1)^3 \binom{\alpha}{3} = (-1)\frac{\alpha(\alpha - 1)(\alpha - 2)}{6}$$

Now:

$$\alpha > 0, \qquad \alpha - 1 < 0, \qquad \alpha - 2 < 0$$

The product of signs becomes:

$$(+)(-)(-) = (+)$$

Applying the leading factor (-1) gives:

$$(-)(+) = (-)$$

Thus:

$$(-1)^3 \binom{\alpha}{3} < 0$$

General Result

For all $k \geq 1$ and $0 < \alpha < 1$:

$$(-1)^k \binom{\alpha}{k} < 0$$

Therefore, every history coefficient inside the summation is negative.

We can rewrite the history summation as:

$$\sum_{k=1}^{H-1} (-1)^k \binom{\alpha}{k} V(t - k\Delta t) = -\sum_{k=1}^{H-1} c_k V(t - k\Delta t)$$

where:

$$c_k > 0$$

Substituting this back into Equation 2.11:

$$V(t) = \frac{I(t) - \frac{1}{\Delta t^\alpha}\left(-\sum_{k=1}^{H-1} c_k V(t - k\Delta t)\right)}{\left[\frac{1}{\Delta t^\alpha} + \lambda\right]}$$

$$= \frac{I(t) + \frac{1}{\Delta t^\alpha}\sum_{k=1}^{H-1} c_k V(t - k\Delta t)}{\left[\frac{1}{\Delta t^\alpha} + \lambda\right]}$$

This shows that, for $0 < \alpha < 1$, the historical membrane potentials contribute as an additive history term where the binomial coefficients may be treated as unsigned values.

Chapter 3

Digital Fractional-Order Dynamics

This chapter aims to provide an understanding of the strengths and limitations of neurons with intrinsic memory in digital hardware from a tradeoff analysis perspective. We begin with an assessment of the design space for implementation on digital hardware of neuron models incorporating intrinsic memory based on fractional-order update dynamics, which revolve around the fractional-order α term, a chosen history length, and resulting binomial coefficients. We focus on the requirements for inclusion of historical membrane potential values, in particular, the bit widths and precision necessary. We then provide an overview of all neuron models designed and tested, which includes a baseline model and two with intrinsic memory. In addition to the GL approximation often used for fractional-order calculations, described in Section 2.2.2, we propose an alternate means of including intrinsic memory as a comparison, using bitshift operations in place of multiplication. We have documented and compared the design considerations for implementation of LIF neurons with intrinsic memory in HDL, serving as a basis for extending their application to full neural networks. This chapter complements existing work by examining in greater depth the design space for HDL implementations of fractional-order neurons, as well as providing a hardware-efficient alternative design for comparison. By doing so, it serves as an engineering-focused foundation upon which future work may be done to evaluate viability of neural networks with intrinsic memory on digital hardware for varying workloads and applications.

3.1 Methodology

We chose to utilize the LIF neuron as a baseline point of comparison across hardware implementations, as it is widely-used and may be considered the current best practice for computationally efficient neuron models, though with limited biological plausibility [14]. The GL approximation technique described in Section 2.2.2 for fractional-order membrane potential update calculations was similarly chosen as it offers a functionally straightforward approach, and as such, is frequently used to model fractional-order equations, particularly in digital applications. [6]

We have implemented three broad categories of neurons:

1. A standard LIF neuron, as described in Section 2.1.1, to serve as a baseline;
2. A fractional-order LIF neuron to test the role of true fractional-order dynamics leveraged for incorporation of intrinsic memory; and
3. A neuron utilizing bitshift operations in place of multiplication as a means of testing inclusion of intrinsic memory in a hardware-efficient manner.

These neurons have been implemented in HDL, simulated, and synthesized on a Nexys A7 board with the Artix-7 100T FPGA.

We chose the Nexys A7 board for a number of reasons, including the on-board peripherals such as seven-segment display, switches, LEDs, and UART, which were useful for basic validation. The Artix-7 FPGA has 240 DSP slices, which was an important consideration as well, since we wanted to have enough DSP slices to map to the individual neurons for their multiply operations once extending our work to full neural networks. Beyond this, the board is well-supported by the Vivado toolchain and has sufficient documentation, as it targets an educational audience.

3.1.1 α (alpha) Choice and Memory Capacity

The intrinsic memory capacity of a fractional-order system is determined by the choice of α value. As seen in Equation 2.8, the binomial coefficient weights that are applied to the historical membrane potential values are determined by α and the history value index represented by k. The relationship between these values is visible in Figure 3.1, which shows that the value of α determines how quickly the binomial coefficient magnitudes decrease with increasing values of k.

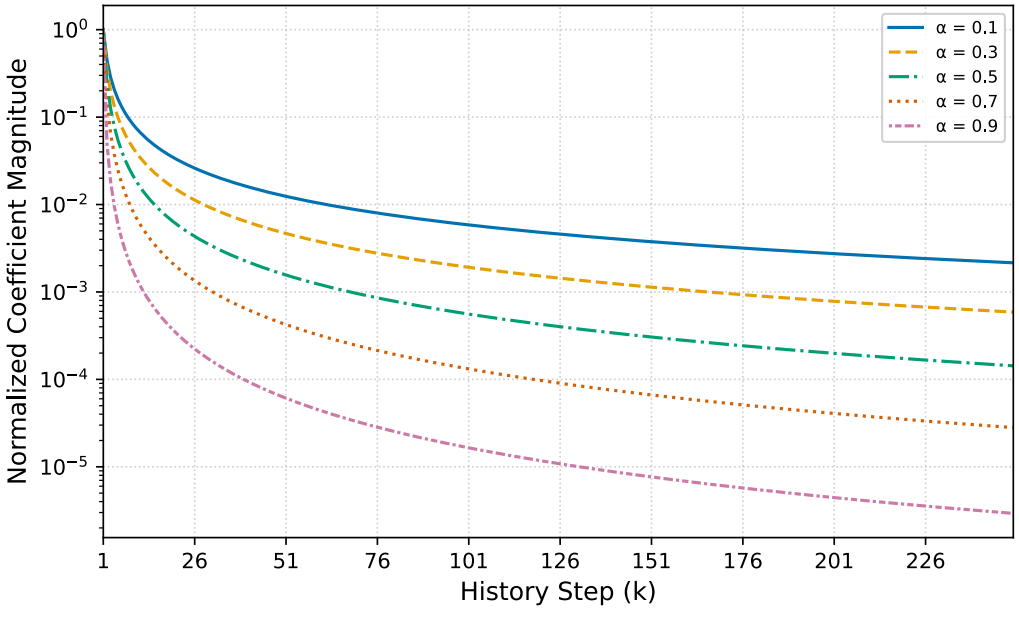

Figure 3.1: Relative coefficient magnitudes for α (alpha) values 0.1, 0.3, 0.5, 0.7, and 0.9. Each series of coefficients is normalized by $\binom{\alpha}{1}$, which is equal to the α value. Coefficient magnitudes of higher α values decay much more rapidly with increasing time step than lower α values. As a result, higher-precision bit width values are required to represent high-α coefficients as compared to low-α coefficients.

In the context of digital implementation, we need to consider the inherent data loss associated with the quantization of values. As the binomial coefficient magnitudes decrease generally with increasing values of α and within a given calculation as applied to historical membrane potential values further in the past, the selection of an appropriate quantization format must be guided by the target α value and history length. Table 3.1 shows the maximum history length across a range of α values for

three different binomial coefficient quantization formats: UQ0.8, UQ0.16, and UQ0.32, where each format is unsigned with zero bits representing the integer component, and 8, 16, or 32 bits, respectively, representing the fractional component. If the bit width of the binomial coefficients is too small for the desired history length, the memory capacity will shrink to match the time step with the binomial coefficient magnitude matching the smallest value representable given the number of bits.

α **Value**	**UQ0.8**	**UQ0.16**	**UQ0.32**
0.1	17	2775	4.18×10^7
0.2	23	2378	2.45×10^7
0.3	23	1643	8.33×10^6
0.4	20	1078	2.97×10^6
0.5	17	699	1.14×10^6
0.6	14	452	4.63×10^5
0.7	11	290	1.97×10^5
0.8	8	179	8.51×10^4
0.9	5	99	3.40×10^4

Table 3.1: Maximum history length (k) across α values given binomial coefficients in fixed-point, unsigned formats UQ0.8, UQ0.16, and UQ0.32. History length decreases sharply as α increases, due to the diminishing magnitude of the binomial coefficients.

Along with the impact on maximum history length, the bit width of the binomial coefficient determines the precision of its magnitude representation, and the resulting calculations when these weights are applied to the historical membrane potential values. Section 2.2.4 shows how the binomial coefficients may be treated as positive values, with the result that the summed history term is added to the input current rather than subtracted. With this in mind, it is possible to utilize an unsigned format for the binomial coefficients and have one additional bit dedicated to precision over the equivalent bit-width signed format. Table 3.2 shows the relative error for the 16-bit quantized approximation of binomial coefficients as compared to their analytical value for a given time step. Binomial coefficients for more recent history terms match precisely to their analytical value for $\alpha = 0.5$, while terms further in the past are

subject to increasing distortion. The cause of the relative error — the decreasing magnitudes of the coefficients — is also a slight mitigating factor in that the history terms impacted receive a smaller weight in the final sum. However, for very long history lengths, increasing precision is desirable.

k **Value**	**Analytical Value**	**Approximation**	**Rel. Error (%)**
0	1.00000000	1.00000000	0.000000
1	0.50000000	0.50000000	0.000000
2	0.12500000	0.12500000	0.000000
3	0.06250000	0.06250000	0.000000
4	0.03906250	0.03906250	0.000000
5	0.02734375	0.02734375	0.000000
6	0.02050781	0.02050781	0.000000
7	0.01611328	0.01611328	0.000000
8	0.01309204	0.01309204	0.000000
9	0.01091003	0.01092529	0.139860
10	0.00927353	0.00927734	0.041135
11	0.00800896	0.00799561	-0.166706
12	0.00700784	0.00701904	0.159902
13	0.00619924	0.00619507	-0.067304
14	0.00553504	0.00552368	-0.205142
15	0.00498153	0.00497437	-0.143881

Table 3.2: Comparison of calculated binomial coefficients for $\alpha = 0.5$ to a 16-bit quantized representation, with the corresponding decimal approximation and relative error. Decreasing fidelity is seen as coefficient values become smaller due to the limitations of the quantized UQ0.16 format.

As discussed above, selection of α value determines a system's history retention as well as the relative weighting of recent historical values. Mastin, et al. [12] provide a formal analysis of these dynamics, noting that lower α values result in high sensitivity to inputs, where the output is heavily influenced by past values, while higher α values create a more predictable and stable model. They state that an intermediate value of α allows for a balanced dynamic, described as computationally powerful. An α value of 0.5, for example, supplies both history retention as well as a predictable response to new inputs.

3.1.2 Baseline LIF

Using the `Leaky` model from snnTorch [8] described in Section 2.2.1 as a functional reference, we created an LIF module in the HDL SV. The LIF membrane update dynamics were mapped into an SV module, with combinational logic used to calculate the membrane potential and spike for the subsequent timestep, and sequential logic used to handle the timing of the output signals' update, as well as reset and clear signal actions. The core calculations were drawn from Equation 2.5, with the multiplication of the decay term, β, by the current membrane potential, the addition of the input current, and finally the subtraction of the reset threshold if a spike had been output in the prior timestep.

3.1.3 Fractional-Order LIF

For our fractional-order LIF neuron implementation, the basic mechanics of applying a leakage term, updating the membrane potential, and determining whether the neuron should emit a spike were similar to the base LIF. Inclusion of the neuron's intrinsic memory of past membrane potential states, the defining feature of the fractional-order LIF, required in-depth examination of Equation 2.9. In particular, we explored the history term seen in expanded form in Equation 2.8 prior to creating a representation in SV in order to ensure numeric representations were of sufficient precision for initial testing and correctly calculating the history term and propagating intermediate values. These implementation details are further elaborated on below in Section 3.1.5.

Behaviorally, we focused on two characteristics indicating fractional-order dynamics which are described by Teka, et al. [16]. The first is upward spike-frequency adaptation, where the inter-spike interval of a fractional-order neuron decreases under constant input current due to the accumulation of the historic membrane potential terms. This is also seen during a constant current with a dropout period, where the inter-spike interval following the dropout is higher than that of the pre-dropout period, also a

result of the intrinsic memory of the neuron. The second characteristic we used for verifying the fractional-order dynamics of our neuron was the display of power law dynamics as opposed to exponential for a neuron experiencing constant current in the subthreshold regime, where the threshold for spiking is set sufficiently high to prevent output spikes from occurring. The expected behavior is the production of a power law curve of the membrane potential output, where lower values of α show an increasingly flattened curve during the input current, and the decay after the input current is terminated produces a flat rather than curved line on a log plot.

We chose to work primarily with a fractional-order α value of 0.5. This value was selected for the benefits described above in Section 3.1, where we describe how $\alpha = 0.5$ results in a balance between memory retention and stability of outputs. Despite the focus on this particular value of α, we ensured our modules were properly parameterized for testing of alternative configurations.

3.1.4 Bitshift LIF

Our goal with the bitshift LIF was to approximate true fractional-order dynamics with reduced DSP slice hardware requirements by taking advantage of bitshift operations in place of multiplication operations involving a fractional binomial coefficient. Our choice to work primarily with an α value of 0.5 noted above in Section 3.1.3, while chiefly related to the unique characteristics of this value, also supported our work in finding a suitable bitshift sequence. This is due to the specific sequence of binomial coefficient values produced for $\alpha = 0.5$. Table 3.3 shows the series of coefficients resulting from a simple incrementing bitshift as compared to the analytical value used in the GL approximation. Though the values diverge, there is overlap between them, hinting at the possibility of a rough equivalence.

We thus began development of a bitshift-based calculation of membrane potential history by exploring four different bitshift patterns and how closely they matched

Index	Analytical Value	Bitshift Value	Rel. Error (%)
0	1.0000000000	1.0000000000	0.00%
1	0.5000000000	0.5000000000	0.00%
2	0.1250000000	0.2500000000	100.00%
3	0.0625000000	0.1250000000	100.00%
4	0.0390625000	0.0625000000	60.00%
5	0.0273437500	0.0312500000	14.29%
6	0.0205078125	0.0156250000	-23.81%
7	0.0161132812	0.0078125000	-51.52%
8	0.0130920410	0.0039062500	-70.15%

Table 3.3: Comparison of calculated binomial coefficients for $\alpha = 0.5$ against a simple bitshift approximation. Relative error is computed with respect to the analytical coefficient values. The bitshift representation follows successive powers of two $(2^0, 2^{-1}, 2^{-2}, \dots)$. While differences increase over time, there are several overlapping values.

the fractional-order binomial coefficients. We categorized the four variant sequences as simple, slow decay, custom, and custom slow decay. In the simple sequence, the number of bits to shift the historical membrane potential value by increased by one with each term. Though straightforward to implement, this variant had obvious issues, such as the rapid decay of the coefficient magnitudes, down to numbers which could not be represented even in software with 64 bits. The slow decay sequence attempted to address this issue by repeating each shift level, resulting in the shift pattern of $1, 1, 2, 2, 3, 3....$ While the issue of the rapid decay was slightly improved, this model still resulted in a relatively poor equivalence to the fractional-order coefficients, with relative errors ranging from 287-540% over coefficients 2-8, and a maximum history length of 32.

The final two sequences were more complex, attempting to maintain greater fidelity to the fractional-order coefficients while resulting in sequences which could be determined programmatically without difficulty. Both of these sequences sought to achieve the highest concordance with the fractional-order binomial coefficient values within the most recent historical terms, which were subject to the largest weighting

in the history sum. As such, both began in the same manner: a shift of 0 once for $k = 0$ (though not formally utilized in the calculation), shift of 1 once for $k = 1$, skip shift 2 as this value was not present in the fractional-order sequence, and shifts 3 and 4 once each. From there the custom bitshift repeated each subsequent shift value a fixed number of times, defaulting to three. This results in the sequence

$$[0, 1, 3, 4, \underbrace{5, 5, 5}_{3\times}, \underbrace{6, 6, 6}_{3\times}, \ldots].$$

The custom slow-decay sequence repeated each subsequent shift value an increasing number of times, equal to $N - 2$ where N is the shift value, resulting in

$$[0, 1, 3, 4, \underbrace{5, 5, 5}_{3\times}, \underbrace{6, 6, 6, 6}_{4\times}, \underbrace{7, 7, 7, 7, 7}_{5\times}, \ldots].$$

Figure 3.2 shows the increasing shift values as compared to the history step, or value of k. Horizontal lines indicate markers for 16 and 32, which are significant for shift operations as they are the point at which the resulting shift output will be zero when applied to any 16-bit or 32-bit operand, respectively. The simple shift has a linear relationship, such that at history step 16, any 16-bit operand will become zero. The alternate sequences cross those thresholds later, with the custom slow decay sequence remaining under the threshold through a history length of 64. The custom slow decay sequence reaches the 16-bit threshold at a history step of 106, meaning it supports a maximum history length of 105 if applied to 16-bit operands. Similar to the choice of a fixed-point format for binomial coefficients for a GL fractional-order approximation, the selection of a sequence here has implications for the maximum history length supported.

Beyond the implications for the maximum history length, the fidelity to the fractional-order binomial coefficient values is another factor we considered when

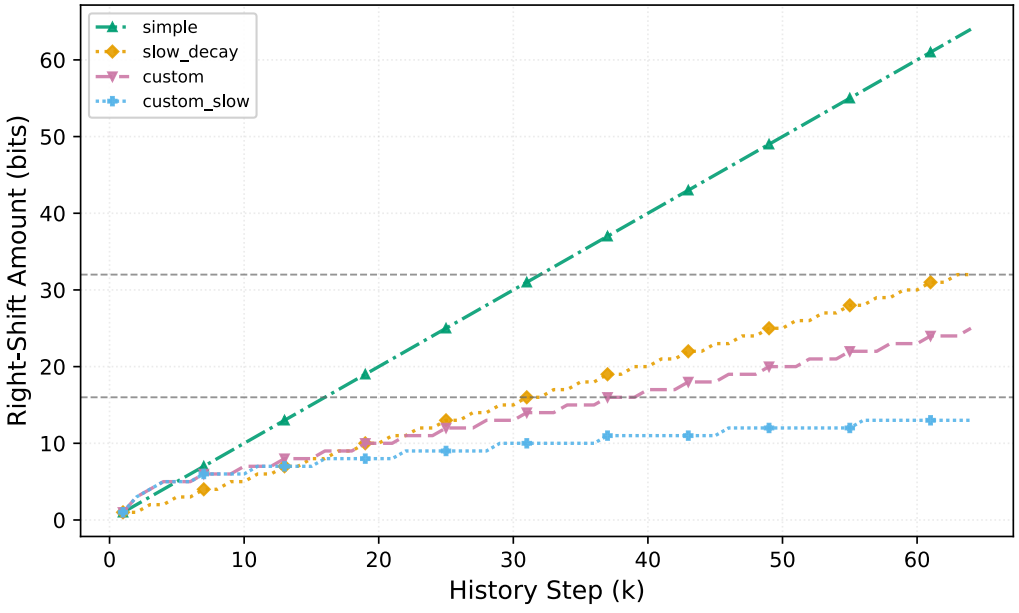

Figure 3.2: Increasing shift values for simple, slow decay, custom, and custom slow decay bitshift sequences as compared to the history step, or value of k. Horizontal lines indicate markers for 16 and 32, which are significant for shift operations as they are the point at which the resulting shift output will be zero when applied to any 16-bit or 32-bit operand, respectively. The simple shift crosses the 16- and 32-bit thresholds earlier than the other sequences, with the custom slow decay sequence remaining under the thresholds through a history length of 64.

choosing a bitshift sequence. Figure 3.3 shows coefficient magnitudes as history step k increases, where magnitude for the bitshift sequences is calculated as $\frac{1}{\text{bitshift value}}$. Included in the plot are the calculated GL binomial coefficients, as well as their corresponding values quantized to an unsigned, fixed-point format utilizing 16 bits.

The quantized and analytical values are nearly identical, reflecting the minimal relative error between the values shown above in Table 3.2. The custom slow decay bitshift sequence is the closest match to the analytical and quantized values. The simple, slow decay, and custom sequences all reduce in magnitude beneath the minimal value possibly represented in a fixed-point, 16-bit format prior to history step 50. This is the equivalent to the point at which the sequences cross the 16-bit threshold in Figure 3.2, where application to a 16-bit operand will result in zero regardless of the operand value.

Regarding behavioral analysis, as we attempted to achieve as close a functional

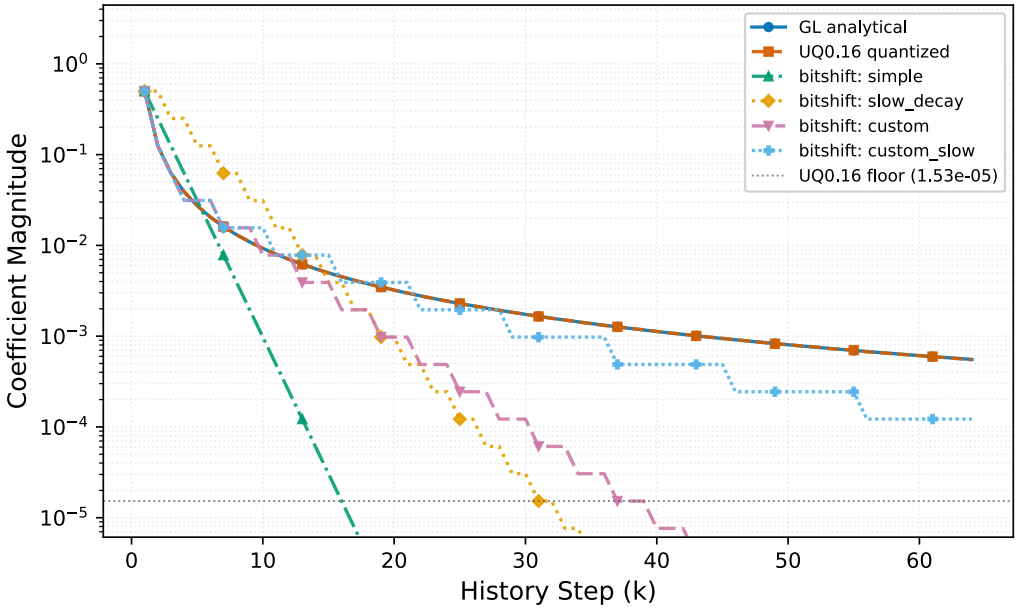

Figure 3.3: Coefficient magnitudes for increasing history step k. Magnitude for bitshift sequences is calculated as $\frac{1}{\text{bitshift value}}$. The quantized coefficient magnitudes used by the fractional-order neuron shows the closest correspondence to the analytical values, with nearly-overlapping plotted values. Of the bitshift sequences, the custom close decay is the closest to the analytical. The three other bitshift sequences cross beneath the floor representation for a fixed-point 16-bit value prior to reaching history step 50.

replication as possible to the fractional-order neuron, we also focused on the characteristics of upward spike-frequency adaptation and power law dynamics for the fractional-order neuron noted above in Section 3.1.3 and originally described by Teka, et al. [16]. Unless otherwise specified, we chose to utilize our custom slow decay bitshift sequence in the following simulation and synthesis, due to its closer fidelity to the analytical binomial coefficient values described above.

3.1.5 Implementation in HDL

As described above, three types of LIF neuron models were created in SV: standard, fractional-order, and bitshift. Tools from the oss-cad-suite toolchain [39] were used for simulation, including cocotb [9], a Python framework targeting chip verification, along with `verilator` and `gtkwave`. Each neuron model had module-specific tests validating basic behavior related to control signals such as reset, clear, and enable, as

well as tests addressing datapath outputs such as spiking and membrane potential values.

Figure 3.4 shows a block diagram of the generalized LIF neuron. This same interface was used for all implementations of our neurons in order to provide a comparable testing environment by allowing the neural network to interface with these neurons in a standardized manner. Though bit widths for the input `current` and output `membrane_out` were parameterized to allow for varying sizes, we ultimately used the same widths matching the defaults for all neurons. This input bit-width of 16 was chosen due to our initial reference point of the snnTorch [8] library's `Leaky` model, which has a default threshold of 1.0 and β decay value of 0.9, resulting in inputs in the 0-2.0 range for varying spike response levels. A fixed-point signed format of sQ2.13 for the input `current` allowed for the typical maximum range expected, and a high level of granularity in the fractional part of the value. For the output `membrane_out`, 24 bits allowed for 13 fractional bits, matching the input, and substantial room in the integer portion of the format to support the accumulation of values necessary in the history term calculations for the fractional-order and bitshift models while preventing overflow. We maintained the parameterization of these bit widths though, to allow flexibility to reduce or increase precision depending on the application.

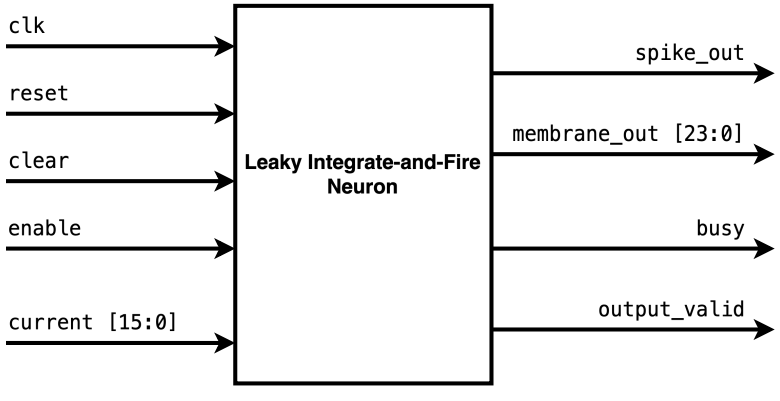

Figure 3.4: Block diagram of a general LIF neuron, with default bit widths for input `current` and output `membrane_out`. All neuron models were intentionally designed to have the same interface, making it easier to test comparable models.

For our fractional-order neuron, we created a script to produce both the calculated, parameterized constants such as the decay term λ, as well as the binomial coefficients for the given α and history length. The coefficients were saved in a .mem file to be read into the module on initialization. The bitshift neuron was similarly supported by a script to produce the necessary constant parameter values.

The base LIF neuron output calculation was simple enough not to warrant a FSM design, with the calculation completing in a single clock cycle. The control signals `output_valid` and `busy` were maintained for this neuron regardless, to support a standardized interface. Both the fractional-order and bitshift neurons required FSMs and multi-cycle computation, due to the history term computation. In order to maintain a high degree of precision, we largely chose to expand bit widths of internal signals for intermediate calculations, with some intermediate truncation through shifting values right and slicing desired bits. Saturation in the bitshift and fractional-order neurons was performed prior to setting the final `membrane_out` output value. While we chose to expand bit widths through intermediate calculations, we did implement parameterized guard bit settings to allow for eventual reduction of bit widths to match actual value ranges encountered.

As described above in Section 3.1.1, after analysis of the fractional-order history term, we determined we were able to increase precision for binomial coefficients by using only the magnitudes instead of signed values.

3.1.6 Synthesis

After passing test results were obtained in simulation, we synthesized the modules using the Vivado Design Suite software [40]. Resulting .bit files were programmed onto a Nexys A7 FPGA board, a moderately-sized board primarily used in educational contexts. This board provided ample resources for synthesizing a single neuron at a time, along with a demo top module. Additionally, we believed it would be sufficient

for synthesis later as we worked to expand to full neural networks.

For each neuron, we created a simple top module and accompanying synthesis script to demonstrate basic functionality. Our top module input a constant current to the neuron under test and incremented a spike counter for each spike output by the neuron. The final count was displayed using the on-board 7-segment displays. An alternate constant current could be selected using an on-board switch. In a Python script, we separately calculated the expected output number of spikes under each condition to ensure the final count displayed matched our expectations.

Through the process of synthesis, we iterated on the original designs for the fractional-order and bitshift neurons. We had initially implemented a higher degree of parallelism in both, specifically in the accumulation of the historical membrane potential values into a single, summed history term. For the fractional-order neuron, this was in the form of parallel MAC operations as each binomial coefficient was applied to each historical membrane potential value from a history buffer. For the bitshift neuron, this was the shift operation applied to each historical membrane potential value, followed by addition. Timing failed for each of these neurons in synthesis using these simple, highly parallel approaches. To address the timing issues associated with the large, parallel blocks of combinational logic, we adjusted the accumulation stage to sequentially multiply or shift, then add one history value per clock cycle. This increased latency for each neuron in direct proportion to the history length, adding to the cost of a high history length target.

3.2 Results

Beyond verification of basic functional requirements such as appropriate response to control signals, our main goal in simulation was to validate that our fractional-order and bitshift neurons displayed characteristics associated with fractional-order dynamics, and particularly, with intrinsic memory. For synthesis, our primary goal

was to validate that our designs could indeed be synthesized onto FPGA hardware in preparation for usage within a neural network. We also wished to have synthesis of individual neurons serve as an early checkpoint on the ability of our future neural networks to scale to many neurons.

3.2.1 Functional Validation in Simulation

Validation of functionality was initially performed through `cocotb` testing executed in `verilator` and `icarus`, with resulting waveforms inspected using `gtkwave`. Figure 3.5 shows the waveform of the LIF neuron's execution of its test suite. The input current of 0.5 can be seen, which produces two output spikes. Calculations of the membrane potential output are also visible.

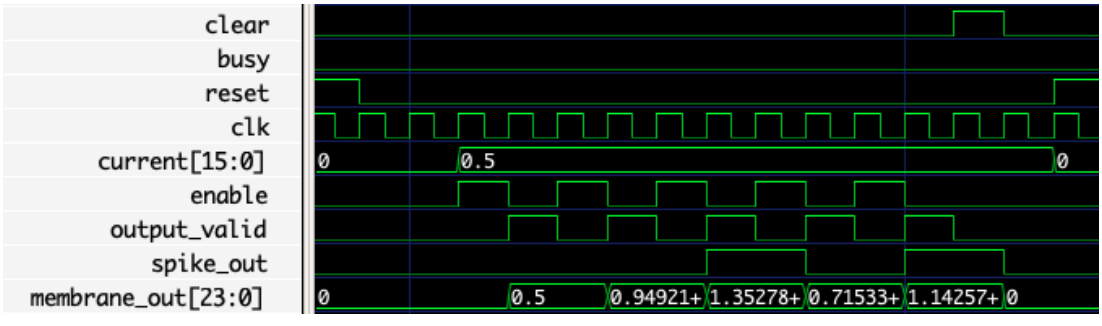

Figure 3.5: Waveform showing the LIF module simulation. A simple, 1-cycle calculation is performed, resulting in the `spike_out` and `membrane_out` values. In contrast to the multi-cycle calculations for the fractional-order and bitshift neurons, the `busy` signal is never asserted due to the single cycle calculation period.

For our fractional-order neuron, figure 3.6 shows a steady input current and accumulation of historical membrane potential values. The `busy` signal can be seen to be asserted for multiple cycles, during which the MAC operation is performed. The index into the history buffer, the history value selected for the current MAC, and the selected binomial coefficient magnitude are all visible.

Similarly, figure 3.7 shows the bitshift neuron performing its shift and add accumulation, during which the `busy` signal is asserted. Shift amounts per cycle and the selected history term, along with accumulated history are visible.

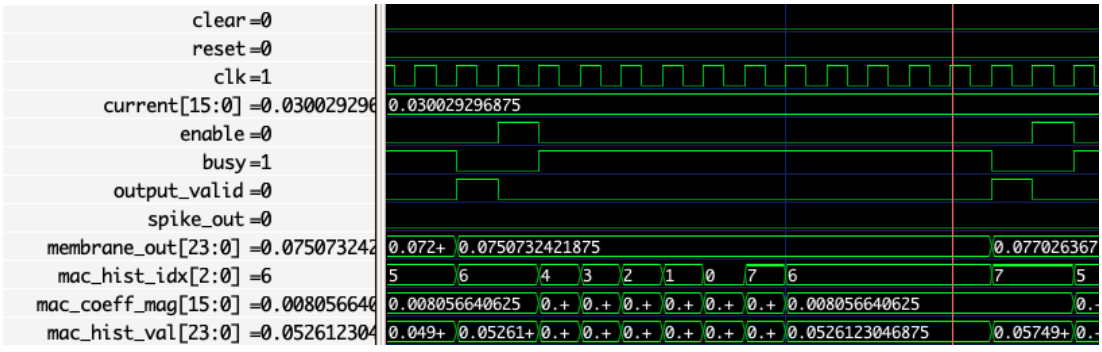

Figure 3.6: Waveform showing the fractional-order LIF module simulation. History terms and indexing are visible, along with the current coefficient magnitude, a quantized value pre-calculated and loaded from a .mem file. The `busy` signal is asserted during the multi-cycle calculation, where the history terms are multiplied by the coefficient values and summed. This calculation adds latency beyond that of the single-cycle LIF.

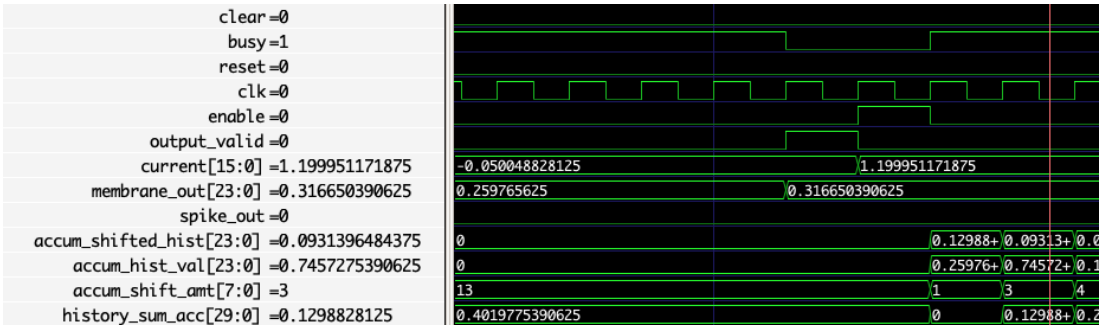

Figure 3.7: Waveform showing the bitshift LIF module simulation. The history value being operated on (`accum_hist_val`) and the bitshift value (`accum_shift_amt`) are visible, along with the prior `membrane_out` result. Similar to the fractional-order neuron, the `busy` signal is asserted for multiple clock cycles to allow for the accumulation of the history values, in contrast to the single-cycle calculation for the LIF.

We verified that our fractional-order implementation produced an increasing spike frequency in the presence of a constant input current, as well as a higher recovery spike frequency after a dropout of a constant input current. These findings are consistent with the behavior described by Teka et al. in [16]. Figure 3.8 shows the increasing spike frequency over time due to the fractional-order system memory, visualized most clearly through the decreasing ISI.

Similar memory effects are seen in Figure 3.9, which shows a higher recovery spike frequency than the initial frequency after a brief period of zero input current.

We were unable to reproduce the upward spike-frequency adaptation in the bitshift

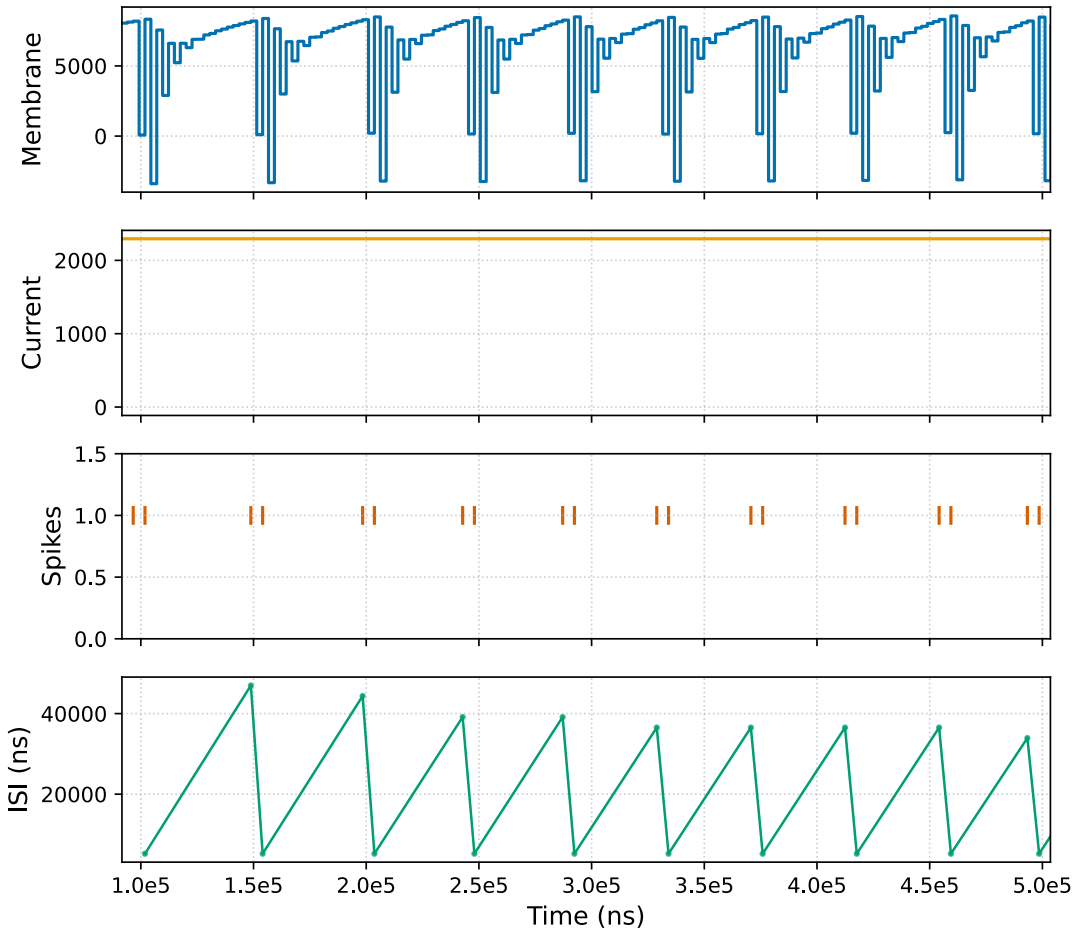

Figure 3.8: Membrane potential over time given a constant input current. In this hardware simulation, the spike-rate adaptation response characteristic of fractional-order systems is visible as the spike frequency increases over time due to memory effects. This is seen most clearly in the decreasing ISI.

neuron using the custom slow decay sequence. However, in addition to the spike-frequency adaptation testing, we also simulated the response of each neuron in a subthreshold regime where the spiking threshold is set high enough to prevent output spikes at a constant input current. The expectation under these conditions is that the decay after the input current is discontinued for a fractional-order response would match a power law curve rather than an exponential curve. Figure 3.10 shows the fractional-order and bitshift neuron outputs plotted on a linear scale up to a maximum timestep of 200, where each neuron had a history length of 256. The shape of the response is the same for each, however, we can see that the bitshift decays much more

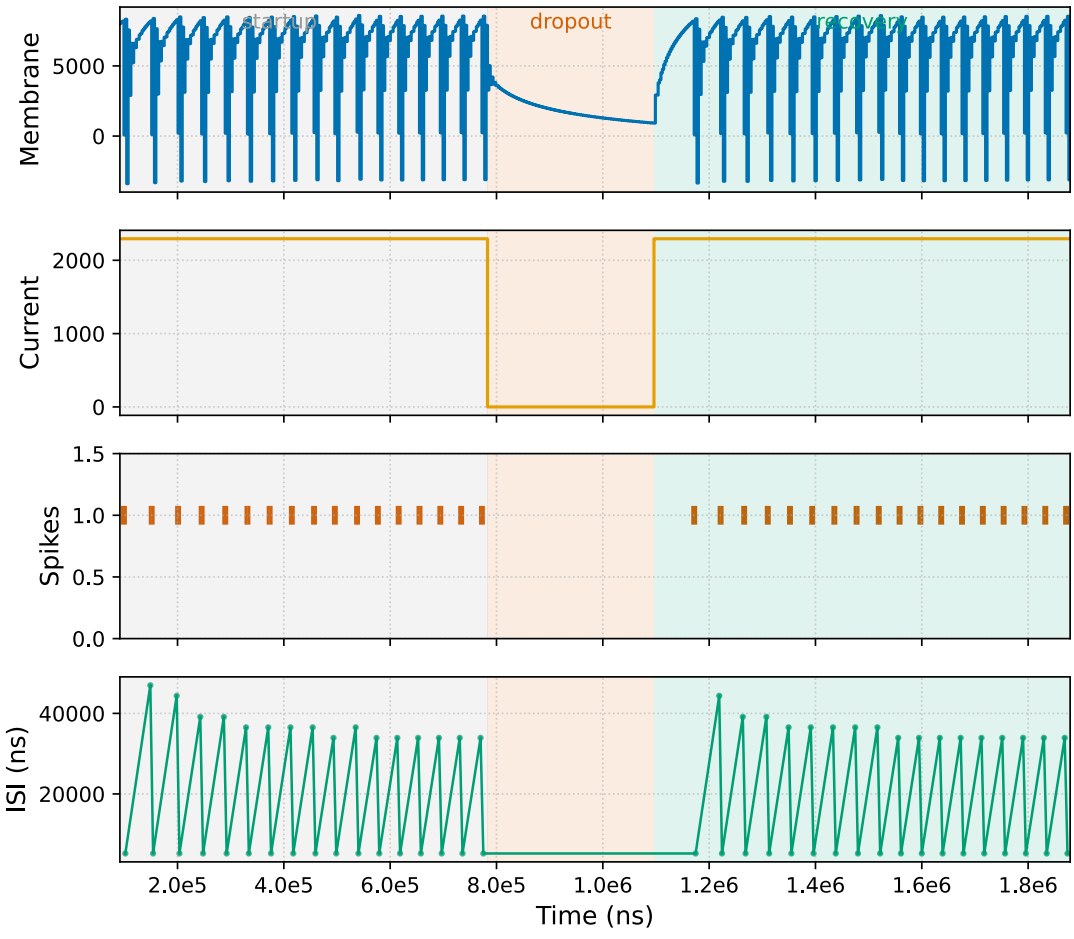

Figure 3.9: Membrane potential over time where input current is held constant until a dropout period of current equal to zero, followed by resumption of a steady current. The ISI following the current dropout is lower than the initial ISI. This behavior is attributed to the intrinsic memory of the system and is characteristic of fractional-order systems.

rapidly than the fractional-order, with divergence beginning around timestep 75.

As exponential and power law curves would be largely overlapping at this number of timesteps, we extended up to 1024 timesteps and matching history length for each and plotted on a log-log scale to highlight differences. Figure 3.11 shows the fractional-order and bitshift outputs on a log-log scale, with exponential and power law best-fit lines matching each. At this extended history length and timestep, the fractional-order tracking to the power law curve becomes much clearer, while the bitshift more closely matches an exponential curve with a faster decay. Taken together, these plots indicate that the bitshift neuron may serve to approximate fractional-order

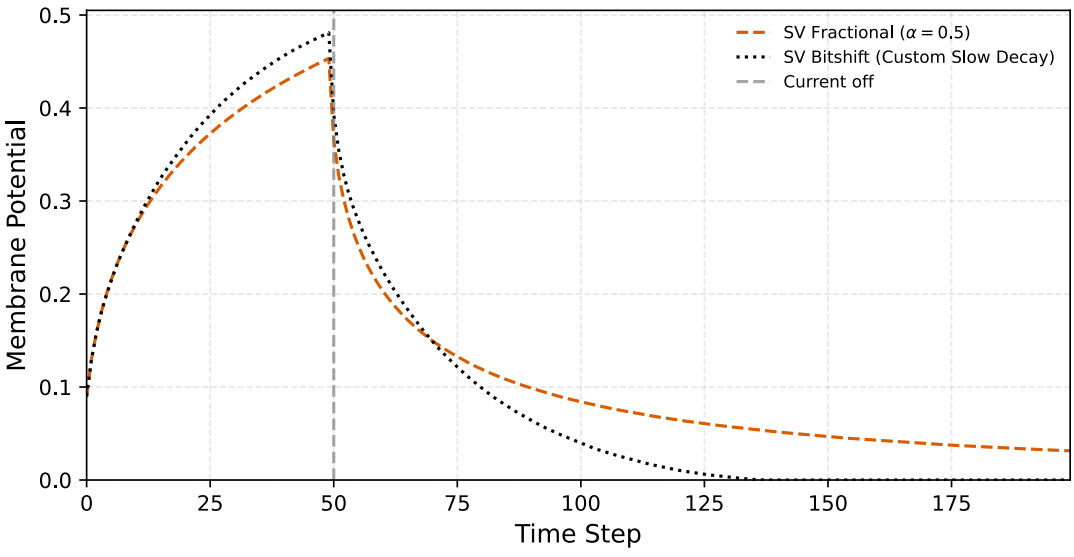

Figure 3.10: The fractional-order and bitshift neurons produce a similar curve in a subthreshold regime where the spiking threshold is set high enough to prevent output spikes at a constant input current. However, the bitshift neuron shows a faster decay to zero after the cessation of the input current. This is a result of the reduced memory capacity of the bitshift neuron, which has a maximum history length of 105.

dynamics at shorter history lengths below ≈75, but the maximum history length of 105 limits its ability to simulate fractional-order dynamics with extended memory.

3.2.2 Synthesis Results

We compiled metrics resulting from synthesis of the three neuron types on the Nexys A7 FPGA with a small top module running a basic demo showing spike count under a constant input current. Table 3.4 summarizes these results, including resource usage as well as performance metrics. As anticipated, the fractional-order and bitshift neurons require more resources total than the standard LIF neuron, due to the overhead of calculating a history term, which includes storage of membrane potential values as well as binomial coefficients or shift level calculation, plus the logic supporting multi-cycle accumulation. The fractional-order neuron utilizes more DSPs because of the MAC operations necessary, while the bitshift requires more Carry4 resources for the carry-chains associated with its shift-add operations.

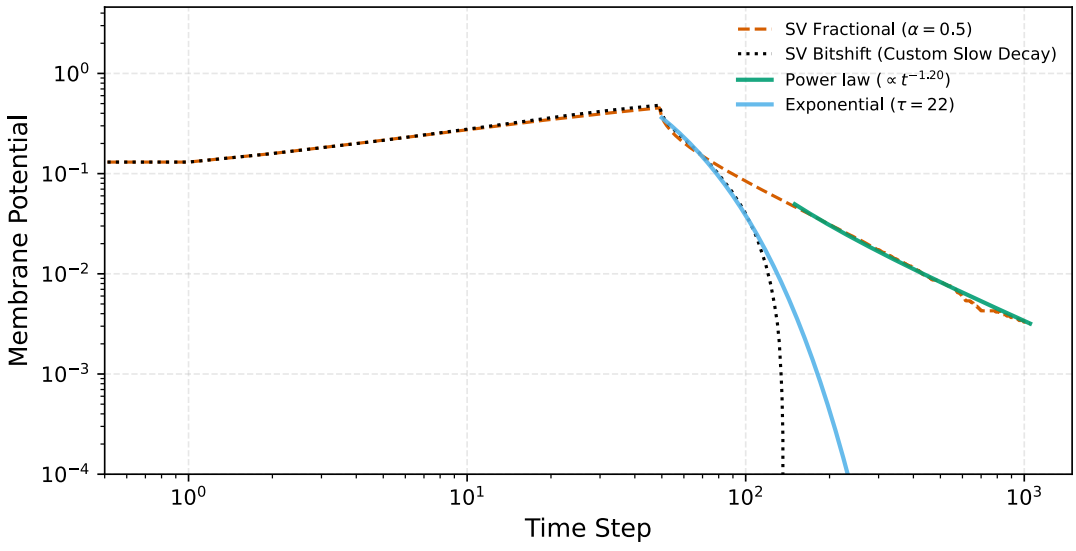

Figure 3.11: Fractional-order and bitshift neurons' membrane potential buildup and decay when subject to a subthreshold input current which is terminated at timestep 50. The log-log axes highlight the fractional-order neuron's power law curve accuracy, where the line largely straightens out and matches a best-fit power law curve. The bitshift neuron, in contrast, drops off quickly around timestep 100, with the overall shape more closely matching a best-fit exponential curve.

The fractional-order and bitshift neurons nearly meet timing for a target 10ns clock period. Based on the calculated F_{max} values being within 1MHz of the target, we chose not to continue iterating to meet timing at this stage, as we felt that premature optimization may be counterproductive as we anticipate further changes will be performed through the process of developing and synthesizing a full neural network. However, we note here that the baseline LIF surpasses timing targets while the fractional-order and bitshift neurons are slightly under-performing. Despite the increased resource usage and calculations performed, the fractional-order and bitshift neurons show comparable total power consumption to the LIF. These neurons do, however, require more clock cycles to compute a new membrane potential output value, so the power consumption is likely higher per calculation despite being reported as slightly lower.

Though an exact comparator for our fractional-order LIF neuron synthesized onto an FPGA was not found in the literature, these results compare favorably with the

Metric	Standard	Fractional	Bitshift
LUTs	93	690	960
Registers	82	1048	1042
Carry4	22	72	130
DSPs	1	5	0
$F_{\max}$ (MHz)	114.81	99.85	99.33
WNS (ns)	1.290	-0.015	-0.067
Total Power (W)	0.101	0.088	0.087

Table 3.4: Synthesis results for three LIF neuron variants targeting a 10 ns clock period. The standard variant meets timing with positive slack; negative WNS values for the fractional and bitshift variants indicate marginal timing violations. Resource usage reflects the added complexity of fractional-order computation (DSP) and bitshift-based approximation (Carry4 chains).

work of Malik et al. [23] in their implementation of a fractional-order HR neuron, also described in Section 2.1.3. Their implementation utilized 12 multipliers per neuron, with power estimates ranging from 316.5–419.25 mW, depending on the approximation technique, where increasing resources and power were required for increasing accuracy in approximation. Our fractional-order LIF utilized five DSPs and an estimated total power of 88 mW. The HR neuron model differs from the LIF in its attempts to more closely mimic behaviors of biological neurons [23], however, we note here that the reduced resources used for our implementation appropriately scale downwards with the decreased complexity of the LIF model.

3.3 Discussion

Our analysis and results show that for a fractional-order neuron using the GL approximation, determining the bit width and fixed-point precision of binomial coefficients is a critical decision which should be informed by the target fractional-order α value and history length. Prior work by Mastin et al. [12] has indicated $\alpha = 0.5$ supports a balance of sensitivity to historical values as well as stability of output. With this in mind, future work may begin with $\alpha = 0.5$ and experimentally determine the history length required for a given task. With these two factors, the necessary bit width

and precision for binomial coefficients may be determined. As such, parameterizing these settings for a neuron implementation is important in maintaining flexibility. As a starting point, our work indicates that 16 bits for representation of binomial coefficients is sufficient for applications using a history length of 100 or less, with an extended history length available when α value is reduced. For very low history lengths, around 17 for $\alpha = 0.5$, an 8-bit format may be sufficient.

For a bitshift approximation of a fractional-order α of 0.5, we found that our custom slow decay sequence minimized relative error from the analytical binomial coefficient values, as compared to three alternative bitshift sequences. This sequence provides a maximum history length of 105 when applied to 16-bit operands, however, fidelity to the fractional-order coefficient magnitudes decreases with increasing history step.

We validated the behavior of our fractional-order neuron by showing fractional-order dynamics in upward spike-frequency adaptation as well as a power law curve response to subthreshold input. Our custom slow decay bitshift neuron failed to show the same upward spike-frequency adaptation. In the subthreshold regime, the bitshift neuron produced a similar response to the fractional-order over shorter history lengths up to $\approx$75. At an extended history length of 1024, the bitshift more closely tracked the exponential decay than the power law curve. This is a result of its maximum history limitations.

Synthesis results confirm that inclusion of intrinsic memory does require more resources, and may present challenges when meeting timing requirements, due to the combinational paths used for history term sum calculation. Latency is also increased for neurons with intrinsic memory, in direct proportion to the history length. For our purposes in this chapter, we chose a simplified approach where a single history term was added to the sum each cycle. However, more complex pipelined implementations may recapture some of the parallelism we removed in order to meet timing for our

single-neuron synthesis demo program. If intrinsic memory does allow for comparable accuracy at smaller neural network sizes, the increased resource usage for intrinsic memory incorporation per-neuron may be offset when scaling these models to usage within a neural network.

3.4 Summary

In this chapter, we have explored the primary design considerations for implementation of neurons with intrinsic memory in digital hardware. We have presented the constraints and requirements for incorporation of intrinsic memory, particularly with regard to fixed-point representation of values such as binomial coefficients. We have proposed an alternative model for calculation of the history term which uses bitshift operations in place of multiplication. This may prove to be more hardware-efficient when resources such as DSP slices are limited, if comparable network sizes may be utilized.

Overall, our work in this chapter demonstrates that history length is a key factor which may serve as both a benefit and a hindrance to performance in digital implementations. The history length may increase accuracy for smaller networks, a possibility we explore in the following chapter, however, we have shown here that increasing history length requires additional resources in the form of expanded bit widths for binomial coefficients, expanded history buffer storage, and a latency increase per calculation.

Chapter 4

Training SNNs for HDL Simulation

With an understanding of the potential benefits and limitations of fractional-order dynamics, we began the process of testing networks utilizing neurons with intrinsic memory in a real-world task. This included our novel bitshift neuron implementation with its approximation of fractional-order dynamics, as an attempt to incorporate intrinsic memory using bitshift operations rather than multiplication. As it is not possible to use traditional training algorithms such as backpropagation with SNNs due to the spiking outputs being non-differentiable [14, 15], we chose to base our development around the snnTorch library, which provides surrogate gradient training methods [8, 38]. The high-level goal with our development process was to be able to train a model on the cart-pole task using neurons with unconventional properties such as intrinsic memory, and then to use a quantized version of that model in hardware simulation, prior to attempting execution on actual hardware. We relied on principles described in work from Mnih, et al. [35] to understand how the experience replay mechanism described in Section 2.1.5 may be used for training in Deep Q Learning, and we referenced examples provided by PyTorch [37] as an applied coding sample of Deep Q Learning.

4.1 Methodology

In order to utilize surrogate gradients as our SNN training method and then leverage the resulting model in hardware simulation, we created a software-to-hardware workflow

using the PyTorch-based snnTorch library [8], custom neuron classes written in Python, quantization and model weight export scripts, and SV modules designed for functional equivalence to their software counterparts. With an end-to-end workflow established, we used the Optuna framework [41] for identifying optimal hyperparameter configurations for neural networks using our different neuron models developed and described in Chapter 3. We then trained models with our optimal configurations and completed our validation flow through to hardware simulation. This laid the foundation for our work in Chapter 5, where we executed our models in hardware on an FPGA.

4.1.1 Algorithm Choice

We chose to utilize a DQN algorithm, as described by Mnih et al. [35], primarily due to it being a value-based method with ϵ-greedy exploration, making it a suitable choice for Q-learning with a discrete action space as we have in the cart-pole task. Additionally, we found the documentation and literature on the DQN approach to be robust, with PyTorch, for example, using DQN in their RL documentation showcasing the cart-pole task [37] and a multitude of articles utilizing DQN or DQN variants on RL tasks in the Atari environment [35, 42, 43, 44].

Specific characteristics of our neural network are supported in the literature as well, such as the decision to use LIF neurons as a baseline, with a shallow, two-layer SNN trained directly using surrogate gradients. Work from Liu et al. [42] utilized LIF neurons in their SNN, which was trained using surrogate gradients, as opposed to using an ANN-trained model converted to usage with an SNN. Though Zanatta et al. [32] use a PPO algorithm, they confirm that SNNs with shallow networks tend to perform better on tasks such as cart-pole, theorizing that this is due to degraded approximation of surrogate gradients at increased depths. Similarly, Sun et al. [43] note that most prior work involving SNNs and RL involve shallow networks.

4.1.2 Workflow for Training and Validation of Models

We necessarily began with software implementation. To this end, we started with a basic training script provided by PyTorch [37], and modified the base script to use an SNN with the snnTorch [8] library by following guidelines from snnTorch documentation [45]. Once we had training in place for the default LIF neuron `Leaky` class provided by the snnTorch library, we created and integrated custom neuron models in Python as child classes of the `Leaky` class.

Our custom neuron classes were based on our fractional-order and bitshift neurons from Chapter 3, focusing on $\alpha = 0.5$ for the fractional-order neuron and the custom slow decay coefficient sequence for the bitshift neuron, described in Section 3.1.4. Similar to our behavioral verification from Chapter 3, we confirmed the power law curve response to a subthreshold input current. Figure 4.1 shows the response of our Python classes for the fractional-order neuron with varying values of α, the bitshift neuron approximating $\alpha = 0.5$, and the `Leaky` neuron from snnTorch, while Figure 4.2 shows this on a log-log scale, highlighting the flat power law response during the decay. As intended, the fractional-order neuron shows flattening of the curve with reduced values of α, with a straightened power law curve response during the decay. The bitshift neuron shows similar behavior to the fractional-order $\alpha = 0.5$ case, but notable deviation from the straight power law curve during the decay phase seen in Figure 4.2. As with our SV hardware implementation, this is likely due to the decreased intrinsic memory, where the history length is capped at a maximum of 105.

With custom neuron classes integrated into snnTorch, we were able to begin training and producing models, with a baseline LIF, a fractional-order LIF, and a bitshift LIF. Following standard practice for RL, our environment was randomly initialized at the outset of each episode, drawn uniformly from $[-0.05, 0.05]^4$ as described in Gymnasium documentation [10]. Machado et al. [46] discuss the principles behind

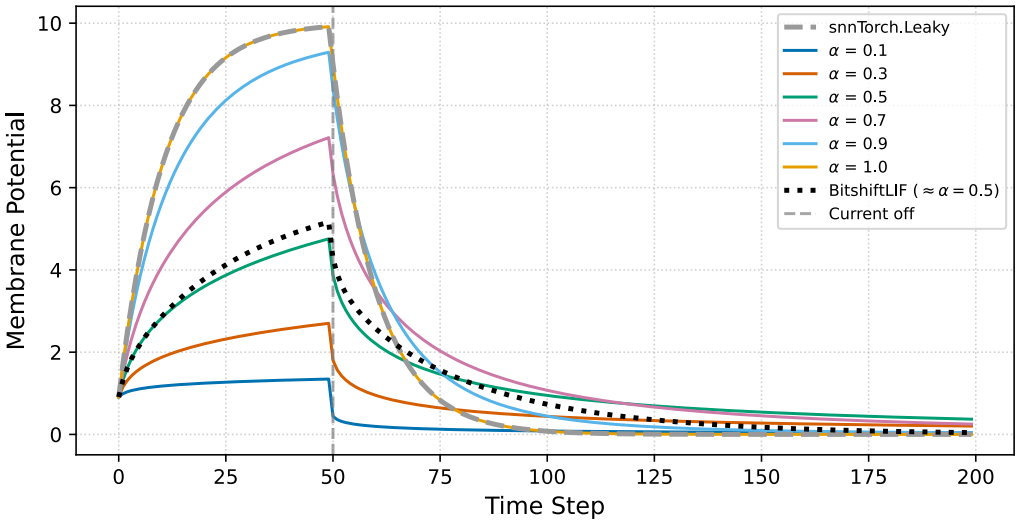

Figure 4.1: Response to subthreshold input current of our fractional-order neuron implementation with varying values of α, our bitshift implementation approximating $\alpha = 0.5$, and the snnTorch `Leaky` neuron, which matches the fractional-order $\alpha = 1.0$ case where classic LIF dynamics are recovered. The bitshift approximation deviates from the fractional-order $\alpha = 0.5$, but does not revert fully to the classic LIF behavior.

this approach, highlighting the benefit of stochastic starting conditions in promoting policy generalization over trajectory memorization. For a given model, we followed the standard in the Gymnasium `CartPole-v1` environment, categorizing a model as successful if it reached an average reward of at least 475 over its last 100 episodes with a maximum episode length of 500 steps [47].

Once we had trained models which could be re-used for inference, we began the process of transforming our models to meet the requirements and restrictions imposed by our intended execution in hardware. To do so, we investigated model weights to determine the best quantization approach, for example, reviewing maximum and minimum values and required precision for representation. Based on this analysis, we generated new quantized model weights from each candidate model. These quantized models were validated using an inference-only script targeting the same cart-pole environment used for training. If a quantized model failed to reach an average reward of 475 over the last 100 episodes of evaluation, an alternative quantization scheme was attempted and the validation inference script was run again. If a quantized model

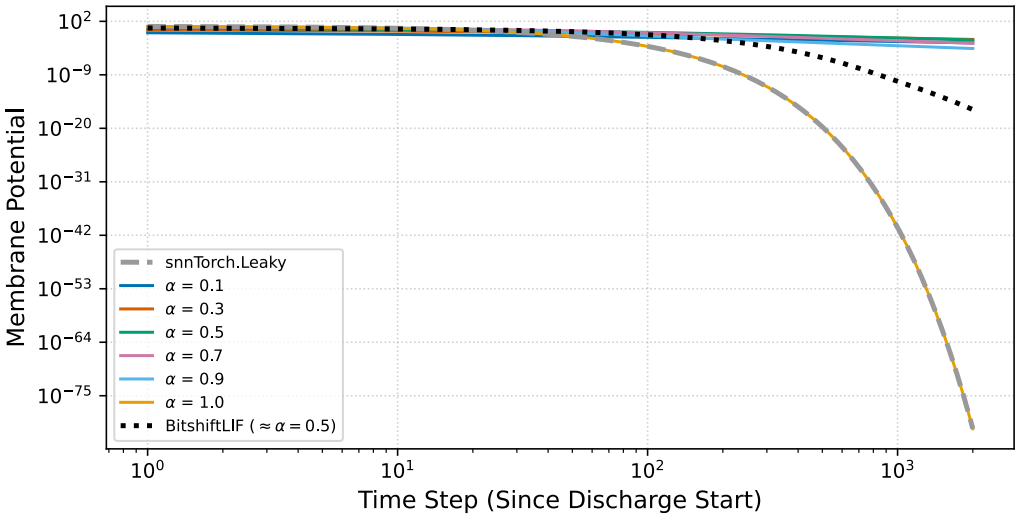

Figure 4.2: Decay period following stimulus of a subthreshold input current to our fractional-order neuron implementation with varying values of α, our bitshift implementation approximating $\alpha = 0.5$, and the snnTorch `Leaky` neuron. Our fractional-order neuron with $\alpha = 1.0$ matches the response of the `Leaky` neuron, showing the recovery of classic LIF dynamics.

was successful in completing the cart-pole task by reaching an average reward of 475 during the last 100 episodes consistently, it was considered a valid model for hardware simulation.

With the software used for training and quantized model validation fixed in place, we developed functionally equivalent SV modules. This proceeded concurrently with model training, and began with evaluation of the section of software we needed to implement in hardware. Listing 4.1 shows the core inference operations performed within the snnTorch `forward` method for the network. In this series of instructions, the input `observations` from the cart-pole environment are first provided to a fully-connected linear layer, which multiplies the inputs by weights and adds bias terms to produce current values serving as input for the neurons in the first hidden layer. The first hidden layer then processes these inputs by using the operations seen in Equation 2.5 in the case of an LIF neuron or Equation 2.10 for a fractional-order or bitshift neuron, with a resulting spike output value. A second fully-connected linear layer receives the spike inputs, and similar to the first, produces current values

which are consumed by the second hidden layer. The second hidden layer produces a spike output in addition to the membrane potential, which is used by the final fully-connected linear layer to produce the accumulated Q value outputs, which are finally averaged over the timesteps and returned. Membrane potential is utilized by the final linear layer to support training with surrogate gradients, following models provided in snnTorch documentation [45].

```
# Simulate for num_steps timesteps with the same input
# each step (rate coding)
for _t in range(self.num_steps):
    h1 = self.fc1(observations)  # current input -> hidden current
    spk1 = self.lif1(h1)         # spike output from layer1
    h2 = self.fc2(spk1)          # pass spikes into next layer
    spk2, mem2 = self.lif2(h2)   # output spikes and membrane of final LIF
    q_t = self.fc_out(mem2)      # decode membrane -> Q-values at this step
    # The Accumulation Strategy
    # out_accum accumulates the Q-values across all timesteps.
    # Since the network processes the same input repeatedly,
    # neurons that respond more strongly to this input will
    # fire more frequently, building up higher accumulated values.
    # After the loop completes, the final Q-values are obtained by
    # averaging: q_pred = out_accum / float(self.num_steps).
    out_accum += q_t

q_values = out_accum / float(self.num_steps)
return q_values
```

Listing 4.1: Inference step performed within snnTorch's `forward` method, with cart-pole environment `observations` as input and the output `q_values` representing the selected action for moving the cart left or right. Here, the fully-connected linear layers are represented at `fc1` and `fc2`, with hidden layers `lif1` and `lif2`. This snippet represents the required functionality for our SV implementation.

As we were constrained to match the software implementation provided by snnTorch, the resulting hardware designs had additional complexities that may have been avoided with a true software/hardware co-design approach. This is discussed further below in Section 4.3.3, and in greater detail in Chapter 5. All individual SV modules were validated functionally through tests developed with cocotb [9], a standard HDL validation framework. Figure 4.3 shows a block diagram of the neural network SV

module we created to replicate the functionality seen in Listing 4.1, focused on the dataflow signals from the cart-pole `observations` to the final `selected_action`.

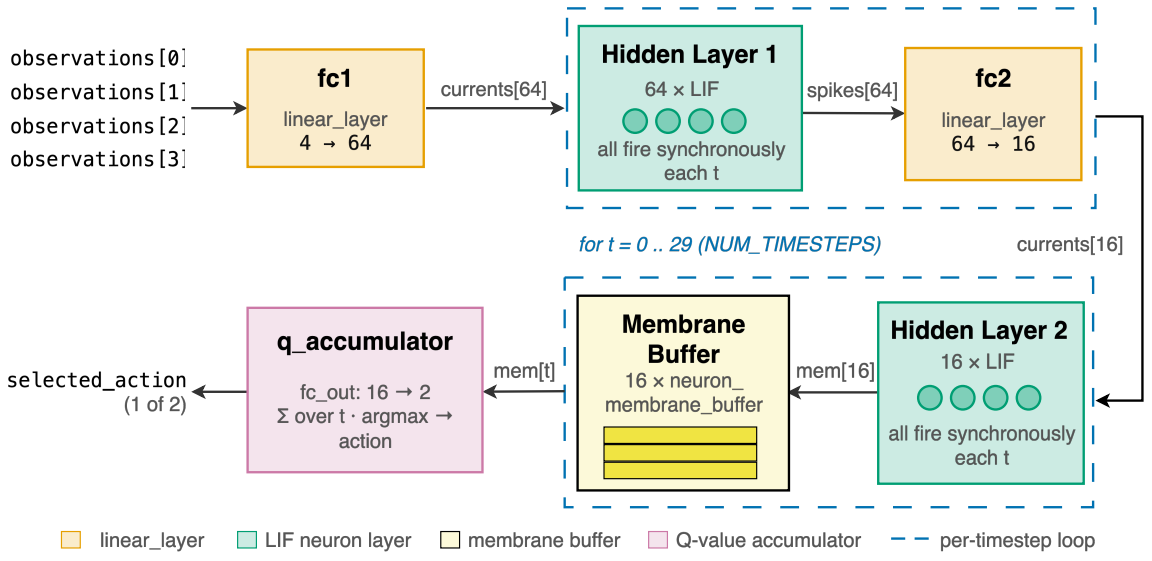

Figure 4.3: Block diagram showing the dataflow of our implemented neural network SV module. Numbers shown here are defaults for a network with hidden layer sizes 64 and 16, using 30 timesteps during which input values are incorporated, reflecting snnTorch defaults. Includes sub-modules `linear_layer`, neuron implementations for each variant, a `neuron_membrane_buffer`, and a `q_accumulator` for final output. These modules replicate the functionality from snnTorch seen in Listing 4.1.

Finally, with quantized, validated models and SV code functionally equivalent to the neural network used for training, we were able to execute the cart-pole task with our chosen models and simulated hardware. We again used cocotb to create a Python-based testing environment for our hardware designs. This served as a crucial integration test used to validate quantized weights and HDL designs prior to targeting execution on actual hardware such as an FPGA.

The entire software-to-hardware pipeline is visible in Figure 4.4, which shows stages classified by execution in software, hardware simulation, or hardware.

4.1.3 Optimization of Models

While the workflow outlined above in Section 4.1.2 provided an end-to-end process for training models for execution in hardware, optimal hyperparameter configurations

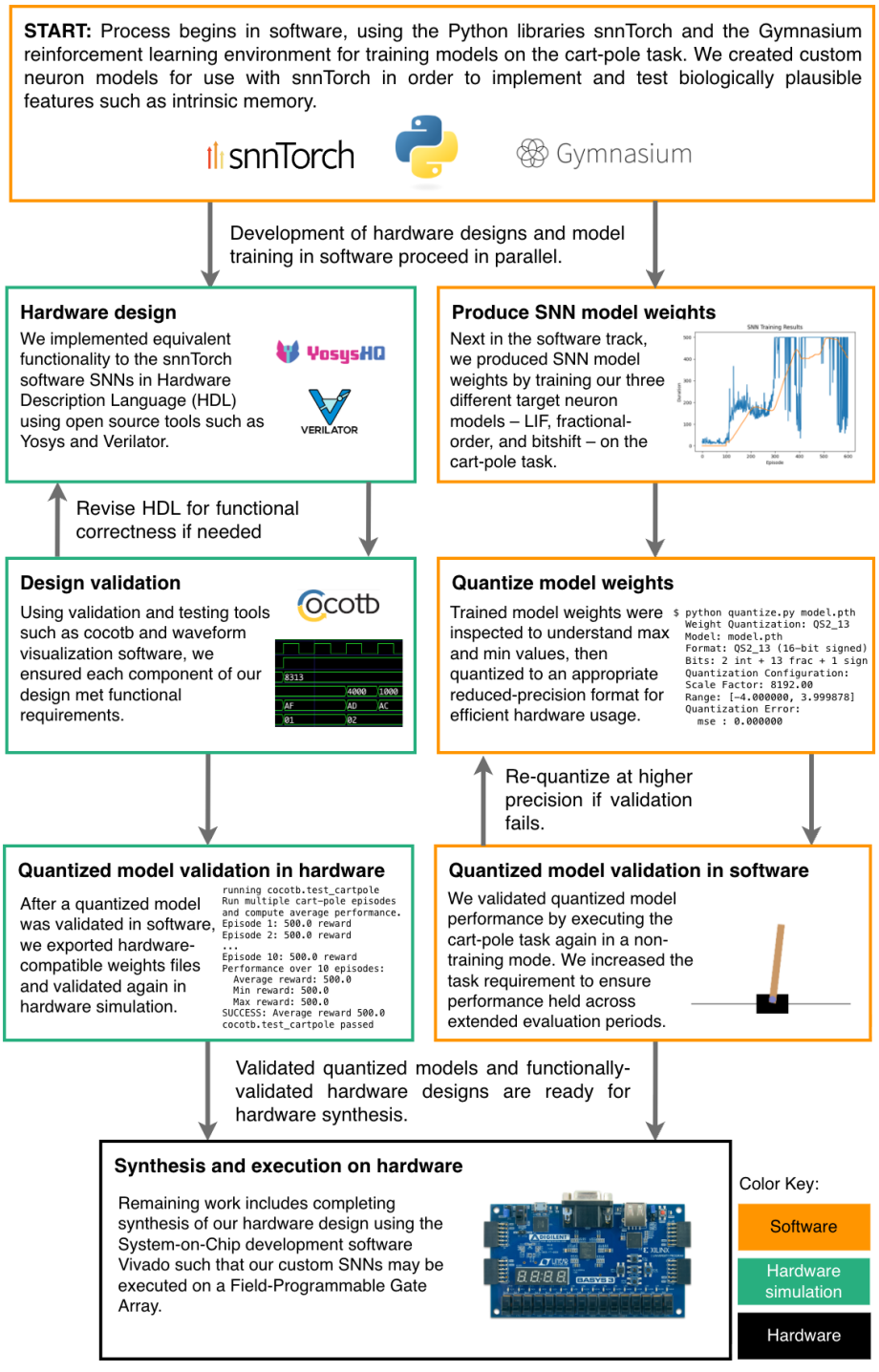

Figure 4.4: Software-to-hardware pipeline, beginning with training in software at top of figure. Functionally equivalent hardware models are then designed in HDL, while model weights may be concurrently generated from the software model. Model weights can then be quantized for more efficient execution in hardware, but must be validated with subsequent inference in software. The final validated, quantized model weights are combined with functionally validated hardware designs to execute on hardware. Example Python script output is shown for model weight inspection, quantization, and validation.

were not yet determined. We utilized the Optuna framework [41] for hyperparameter optimization in order to target candidate configurations we could use for training models using our different neuron types. Following the approach used by Mastin et al. [12], we initially chose to run each training run within an Optuna trial for 2500 episodes. This longer training budget theoretically allowed configurations that converge slowly to be identified, particularly those with smaller networks or longer history lengths that may require more episodes to reach the success threshold. In order to favor models with smaller neural networks, and in the case of the fractional-order and bitshift neurons, shorter history lengths, we added a small penalty based on network sizes and history lengths. This served as a tie-breaker, such that if any two models achieved the same objective score on the cart-pole task itself, the one with a smaller network size or history length would be ranked slightly higher. The penalty for neural network size was calculated as $0.05 \times total\_network\_size$, and the penalty for history length was $0.01 \times history\_length$, and was subtracted from the objective score.

For our objective within each Optuna study, we followed the principle described by Agarwal et al. in [48], where they discussed usage of the IQM as a measure for reporting RL results. While Agarwal et al. [48] applied the IQM across starting seeds, we applied it to a single run as a measure of robustness, applying the IQM across the tailing 25% of episodes. We chose this as our stability metric for our Optuna study objective as it was less susceptible to fluctuating model performance near the end of each training run than a simple final average calculation. The goal was to prevent a case where a model happened to reach a localized peak in performance near the end of the training run which represented a "lucky streak" rather than true convergence. We ran at least 100 trials in three different Optuna studies, one for each neuron type.

One challenge we faced through this process was in the runtime for our studies. To mitigate this challenge, we developed scripts to run multiple processes for a given

study, with the option of targeting either the Central Processing Unit (CPU) or GPU on a system, and specifying the number of workers to use. This allowed us to maximize utilization of resources on a given system and reduce total time to obtain study results.

An additional challenge we faced while gathering preliminary results was that we were seeing models which showed catastrophic forgetting, a phenomenon described by McCloskey and Cohen [49] where new learning in a network results in failure of a model to maintain earlier performance. To address this issue, we attempted multiple training runs on a given hyperparameter configuration with a set starting seed with specific, targeted modifications. We first tried increasing the size of the buffer used by the experience replay mechanism previously described in Section 2.1.5, theorizing that a converged model would eventually have an experience buffer with only near-equilibrium samples to continue training on. Though adjusting the buffer size changed performance, it did not result in a consistent reduction of the catastrophic forgetting behavior.

We next tried updating the target network only every few environment steps instead of every step, more in line with the hard update used by Mnih et al. [35], though in contrast to the usage of the soft update described by Lillicrap et al. [36] and outlined in Section 2.1.5. This resulted in significantly degraded performance overall, and this change was reverted.

We finally adjusted the `weight_decay` setting on the PyTorch `AdamW` optimizer (which the official PyTorch DQN tutorial [37] uses by default) to have a value of zero rather than the default of 0.01, attempting to provide a closer match to the original RMSProp optimizer used by Mnih et al. [35]. With this adjustment, the catastrophic forgetting in our test case was resolved and we saw convergence in this model. Operationally, this change resulted in more stable weights in the target network, as they were no longer being subject to the decay term, which appeared to be a better match for the training required in the cart-pole task.

We were limited in our ability to perform additional configuration adjustments and testing due to the compute time required for each test, which varied from around 12 hours to several days, depending on the specific change, as verifying a change involved running a simulation long enough to see convergence followed by enough episodes to allow for a subsequent collapse in performance to emerge. Additionally, full Optuna studies with 100 or more trials required several days at a minimum, with upwards of a week or more for trials with episodes of 2500, where episode number is fixed across an Optuna study. As such, we adjusted the total episodes per trial down to 1500 after adjusting the `weight_decay` setting, as a practical response to having limited time for finalizing adjusted Optuna studies in time for analysis, multi-seed retraining, and HDL synthesis with model weights in Chapter 5. We feel that 1500 episodes is sufficient for identifying hyperparameter configurations which show convergence on our success metric of an average reward of 475 over 100 episodes, based on our initial testing and finding that convergence occurred in some cases at around episode 900-1000. We discuss this further in Section 4.3.

4.1.4 Identification and Training of Core Models

Once our Optuna studies had completed, we utilized scripts to query each Optuna study database and identify characteristics of passing hyperparameter configurations, as well as the top high-performing configuration for each neuron type. For targeting configurations, we gated on convergence episode, requiring at least the 100 final episodes to have an average reward >= 475 in order to identify configurations which resulted in a true success in solving the task rather than a short-term "lucky" spike. From there, we ranked by IQM to identify configurations with consistently high performance within the last 25% of episodes.

With our candidate configurations identified from the Optuna results, we began training models. In analysis from Colas et al. [50], they show that re-training with

only five seeds is insufficient to minimize the impact of performance variance across starting conditions, and that $N \geq 20$ is necessary for sufficient power in detecting effect size. We therefore chose to execute training on 30 different seeds to evaluate model performance. We worked around the compute time issue in this case by matching our approach in running Optuna studies, and parallelizing the training runs so that training on each seed ran concurrently. We selected the highest-performing seed's resulting model, where selection was first gated on having solved the task by achieving a reward of at least 475 over the last 100 episodes and performance was measured by the IQM.

With the trained models, we proceeded with the process described above in Section 4.1.2, where we quantized model weights and validated performance first in software and then in hardware simulation.

While testing our models in execution on FPGA as part of our work in Chapter 5, we determined that our LIF model was not robust enough to consistently reach a reward of 500. We revisited the candidate models selected from our Optuna studies as described above, and executed multi-seed training for 30 seeds on the next-ranked candidate configuration from the Optuna study. This configuration achieved higher performance with greater consistency, and was used for comparison between models in Section 4.2, in addition to execution on FPGA implemented through our work in Chapter 5.

4.2 Results

Relevant findings regarding neural network characteristics were available in analysis of the Optuna studies as well as in the multi-seed training of candidate hyperparameter configurations. We present these results below, followed by our results from hardware simulation of a full, end-to-end integration test of the cart-pole task using cocotb [9] and quantized weights from a model trained in software.

4.2.1 Optuna Results

Our Optuna studies for each neuron type contained at least 100 completed trials. Number of trials varied slightly depending on how many trials were pruned (e.g. if a constraint was broken, such as if hidden layer one was configured to be smaller than hidden layer two), but all studies were initiated to run for 125 trials. For this analysis, we gated trials as having achieved a "passing" classification if they had achieved an average reward of at least 475 over the final 100 episodes of the trial. In analysis, we added a categorization for trials which achieved an average of 475 over a number of episodes less than our 100 cut-off as well. These two categories are distinct from trials which never achieved convergence by never reaching and maintaining an average reward of 475.

Figure 4.5 shows neural network size versus tail IQM average reward, the IQM over the last 25% of episodes metric referenced above in Section 4.1.3, across all Optuna studies. Optuna's hyperparameter tuning has resulted in the majority of converged fractional-order models utilizing 40 neurons in total between two hidden layers (10 trials), with five trials utilizing 72 neurons. The majority of converged bitshift models (15) required 96 neurons, while seven of nine passing LIF models used 64 neurons. One notable exception is the single bitshift model which utilized only 12 neurons while reaching convergence. The model, however, achieved a lower IQM than most of the models using 96 neurons, and as such, was not selected for the multi-seed training described in Section 4.2.2.

Figure 4.6 shows history length versus tail IQM average reward for the fractional-order and bitshift networks. Convergence was seen entirely at history lengths of eight or less, indicating that an extended history length was not beneficial for model performance.

In total, the bitshift networks showed the highest rate of convergence across trials

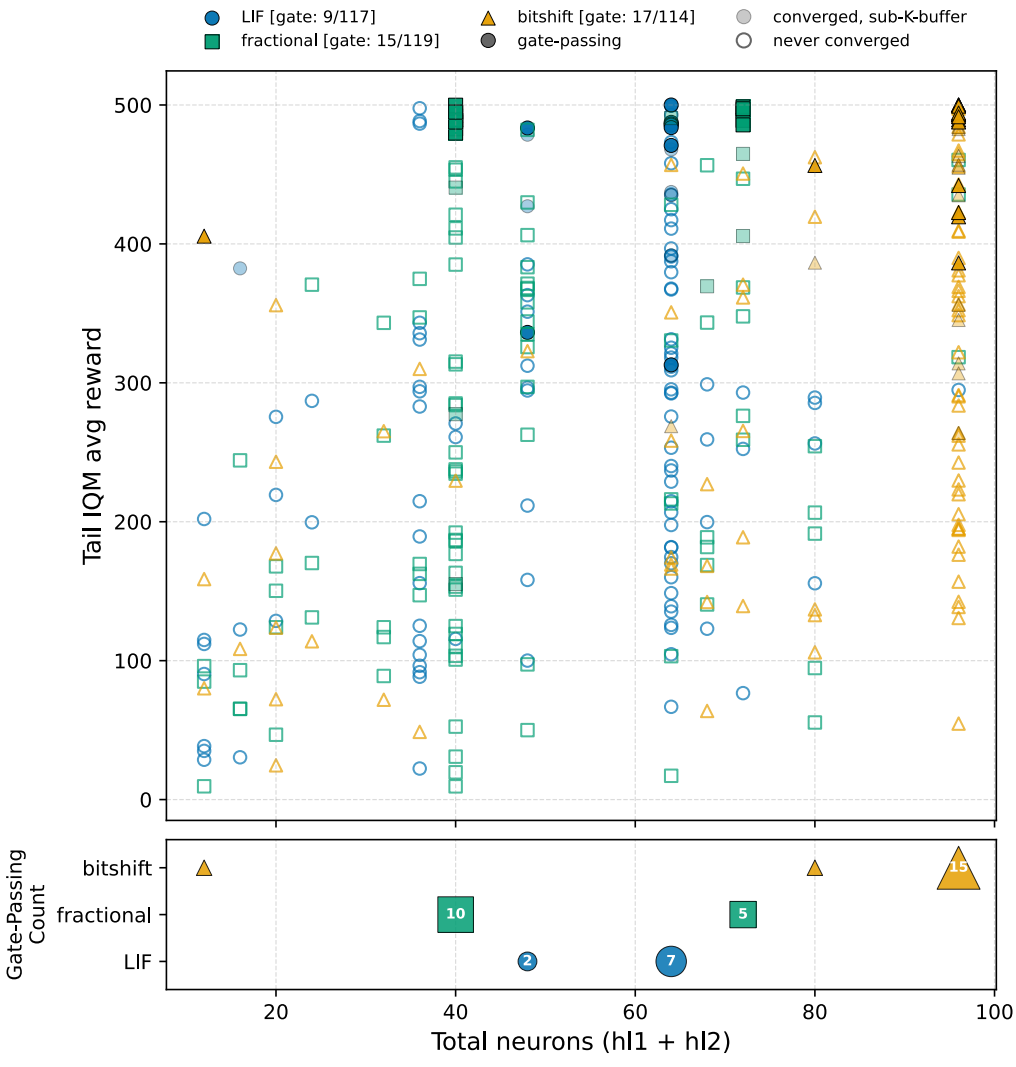

Figure 4.5: Total neural network size (hidden layer one plus hidden layer two) versus IQM of the last 25% of episodes across all trials for three Optuna studies, one for each neuron type. Trials which converged are represented by symbols fully filled in with color, while trials which never converged are empty; trials which reached our convergence-point reward of 475 after our buffer of K episodes are filled in with reduced opacity. Most fractional-order models used either 40 or 72 neurons total, with the majority using 40. The LIF models largely used 64 neurons, and the majority of the bitshift models used 96 neurons, with the exception of a single model using only 12.

at 14.9%, with the fractional-order ranking next at 12.6%, and the LIF network having the lowest rate of convergence at 7.7%. Figure 4.7 shows the converged trials for each neuron type, categorized into converged with fewer total trials than our cut-off $K = 100$, or simply converged, having reached and sustained an average of 475 from episode $Total\_episodes - K$ on to the end of the trial. Convergence episodes as

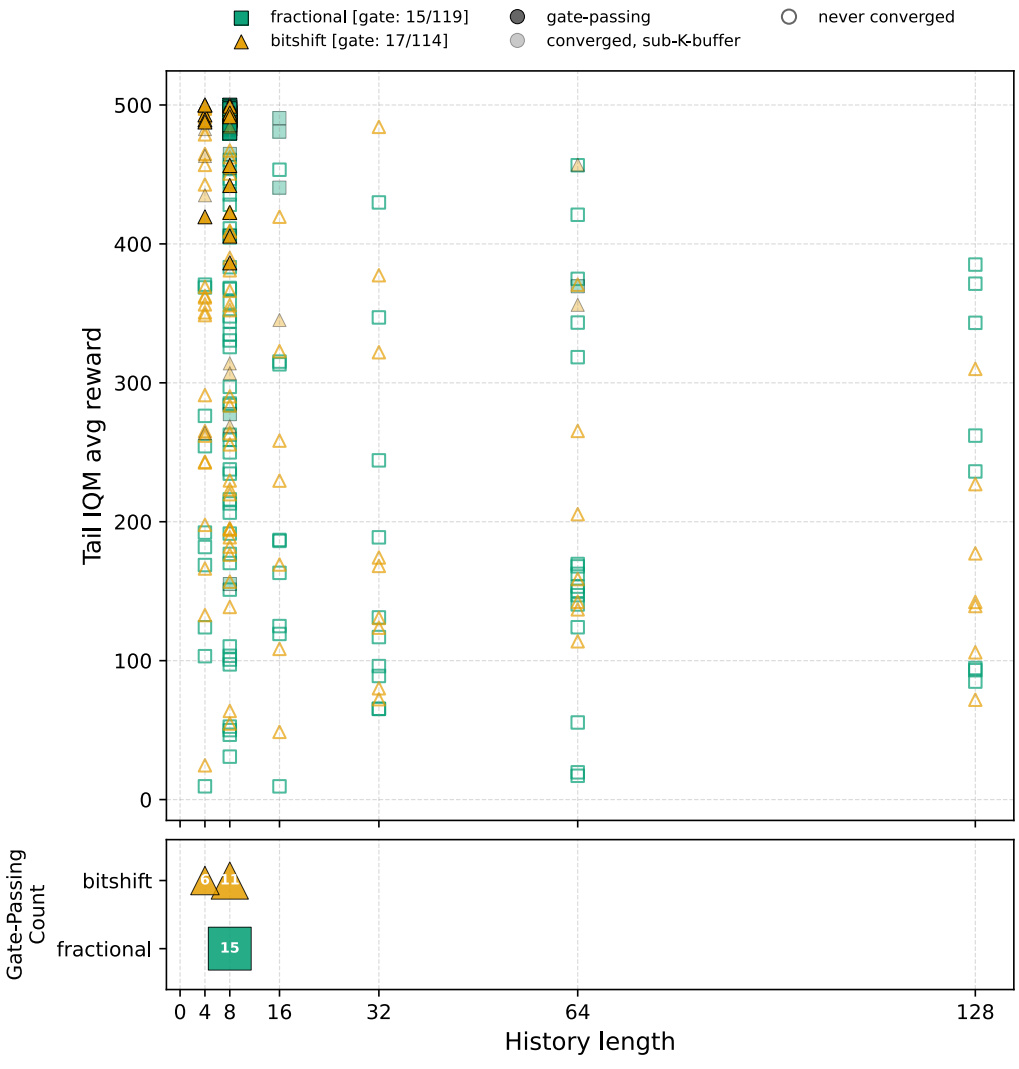

Figure 4.6: Intrinsic memory history length versus IQM of the last 25% of episodes across all trials for three Optuna studies, one for each neuron type. For both the fractional-order and bitshift models, only minimal history lengths of four or eight produced convergence in models, indicating that extended history lengths were minimally beneficial to performance.

early as around episode 800 were seen for the LIF and bitshift types. In contrast, convergence episodes clustered much later for trials using the fractional-order neuron.

4.2.2 Multi-Seed Training

As described in Section 4.1.4, we selected the top configuration for each neuron type from the results of the Optuna studies, as determined by tail IQM. Table 4.1 shows the hidden layer sizes and history length for the selected configuration for each neuron

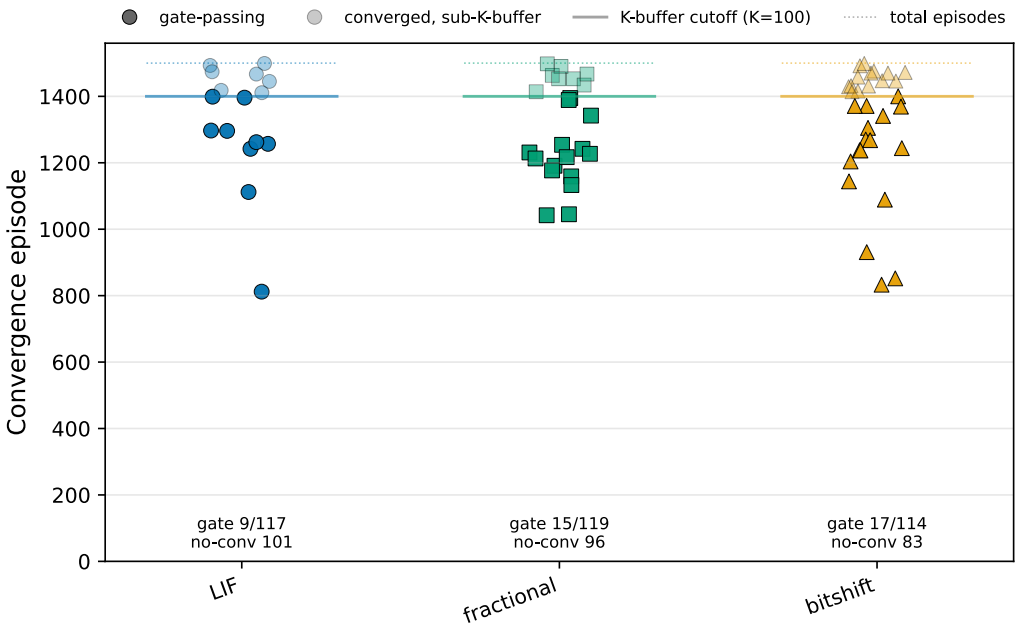

Figure 4.7: Converged trials for each neuron type, categorized into converged with fewer total trials than our cut-off $K = 100$, or simply converged, having reached and sustained an average of 475 from episode $Total\_episodes - K$ on to the end of the trial. The bitshift networks showed the highest rate of convergence across trials at 14.9%, with the fractional-order ranking next at 12.6%, and the LIF network having the lowest rate of convergence at 7.7%.

type. Selection was made based on criteria outlined in Section 4.1.4. We can see that the fractional-order model had the smallest network size, at a total of 40 neurons across two layers, versus 64 and 96 neurons each for the LIF and bitshift models, respectively. The fractional-order model utilized a slightly longer history length than the bitshift, but both were relatively small given the maximum possible of 128.

We executed training for each configuration on 30 starting seeds. Figure 4.8 shows the IQM for each configuration with individual seeds and an Inter-Quartile Range (IQR) band, along with the solved threshold of 475.

We can see that the bitshift configuration resulted in the best overall performance, with the highest average final reward across all starting seeds, and the smallest IQR. The LIF configuration achieved nearly the same level of performance. The fractional-order configuration performance was the least consistent, showing some similarities to the LIF configuration with regard to wide IQR, though with a notable

Parameter	LIF	Fractional-order	Bitshift
Hidden layer 1 size	32	32	64
Hidden layer 2 size	32	8	32
Total neurons	64	40	96
History length	N/A	8	4

Table 4.1: Comparison of selected architectural parameters for the optimized LIF, fractional-order, and bitshift neuron configurations. Values shown correspond to the highest-performing configurations under the evaluation methodology described in Sections 4.1.3 and 4.1.4. While the fractional-order neuron appeared to show some advantage with regard to reduced network size, the bitshift neuron did not produce a similarly-sized model, and in fact produced a larger model than the LIF.

decline in performance around episode 1200. We discuss the implications for training stabilization further in Section 4.3.2.

4.2.3 Hardware Simulation

Using cocotb [9] and Gymnasium [47], we developed a cart-pole integration test to simulate the neural network we had developed to match the snnTorch implementation. In this integration test, the cart-pole state observations provided by the Gymnasium environment were converted from floating point to fixed and set as inputs on the `neural_network` SV module. Once the `done` signal was asserted, the test environment read the `selected_action` output. Figure 4.9 shows the waveform of execution of a single run of up to 500 steps.

Within our cart-pole integration test for hardware simulation, we verified single-inference steps, a 500-episode inference-only sequence with a fixed seed, and 500-episode inference-only sequences across varying start seeds. This confirmed our chosen models with quantized weights were capable of successful completion of the task across varied start conditions.

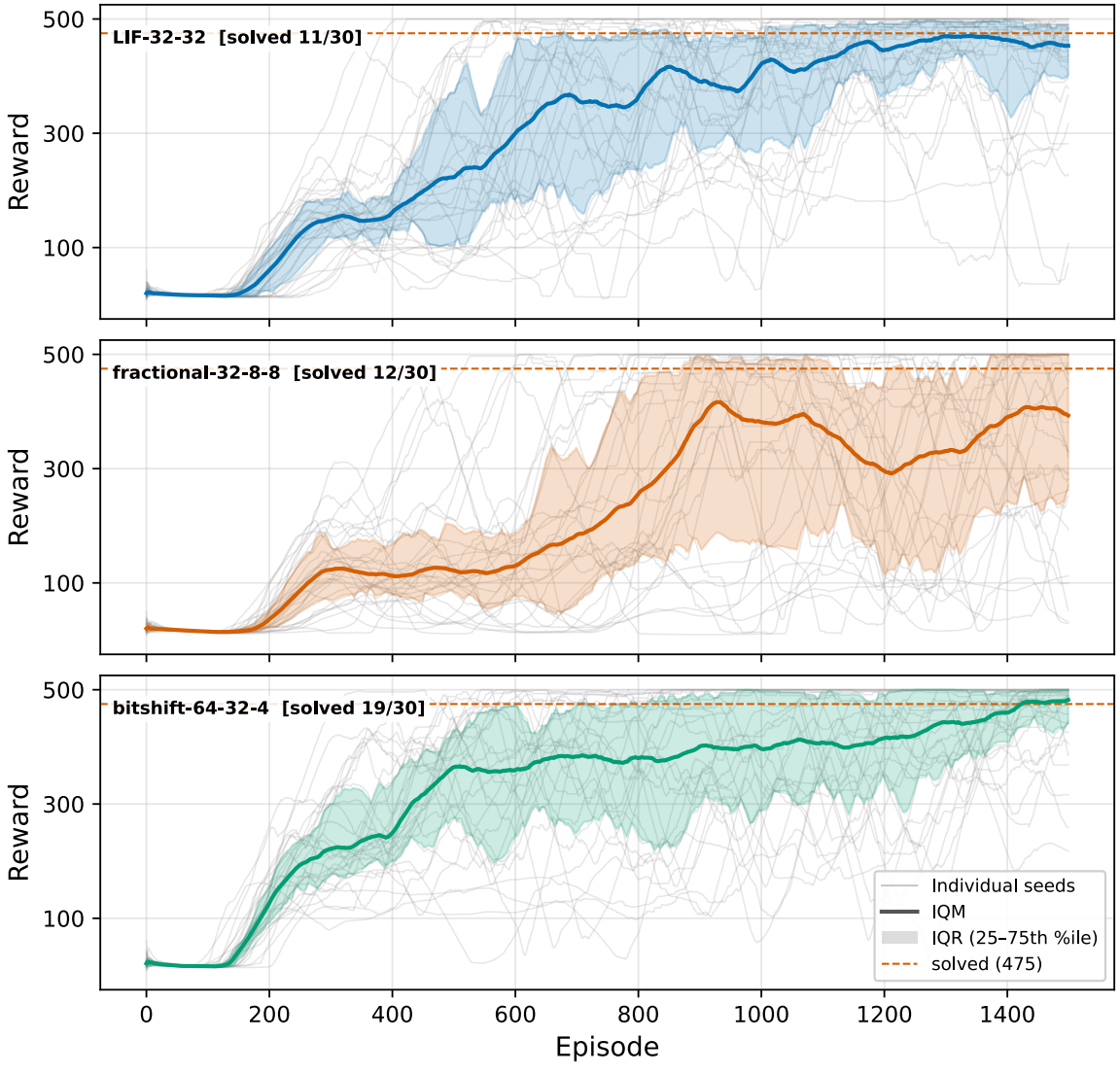

Figure 4.8: Multi-seed training with average IQM in bold across three configurations, one for each neuron type. Each configuration was trained 30 times, each with a different starting seed. The bitshift configuration showed the greatest stability over time, and the highest average reward and solved rate. The LIF configuration showed the lowest average reward and solved rate, while the fractional-order was in between. The fractional-order shows a notable regression around episode 1200, which is discussed further in Section 4.3.2.

4.3 Discussion

Through our process of developing a software-to-hardware workflow for SNN models trained on the cart-pole task, we found that neuron type did have an impact on total network size, where a fractional-order network was able to solve the task with fewer neurons than the LIF or bitshift networks. This aligns with our initial hypothesis that

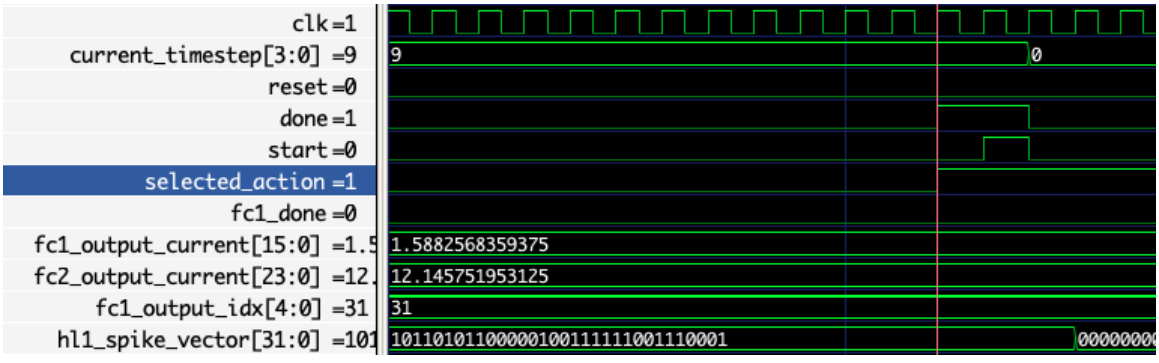

Figure 4.9: Waveform showing simulation of cart-pole integration test. The `selected_action` output is visible, which indicates the right or left direction for the next cart movement. The signals `fc1_output_current` and `fc2_output_current` represent the output current from fully-connected layers one and two, respectively, calculated using spikes and model weights. The signal `hl1_spike_vector` represents all of the spikes in a given timestep from hidden layer one. This vector is 32 bits long, matching hidden layer one's size of 32 neurons.

intrinsic memory allows for greater efficiency with smaller neural networks, though the impact appears to be specific to true fractional-order dynamics rather than the approximation provided through our bitshift implementation. There were a number of additional findings and recommendations for future work based on the results from work presented in this chapter, including possible avenues for further testing in software, training SNNs using DQNs, and implications for hardware development with our approach.

4.3.1 Further Testing in Software

While we intentionally limited our choice of the fractional-order α value to 0.5 in an effort to allow a bitshift operation to better approximate the binomial coefficient values, and due to the properties of this particular α value described in Section 3.1.1, a more expansive exploration of the impact of varying α values may be desirable. Lower α values, for example, may provide opportunities for incorporating longer history lengths than higher values for α, based on the different binomial coefficient magnitude decay patterns seen in each.

4.3.2 Training Inconsistencies

Our challenges in training were primarily seen in the catastrophic forgetting and sometimes chaotic training patterns described above in Section 4.1.3. We found that reducing the `weight_decay` setting on the AdamW optimizer to 0 resolved our issues with catastrophic forgetting on the targeted tests we attempted. This was likely due to relieving the added pressure towards overall weight decay, which accentuated the decay imposed by the bootstrapping of targets in training using the target network described in Section 2.1.5. Additionally, the experience replay buffer, also described in Section 2.1.5, naturally loses transitions the model has sampled early in training, and the decay may overly suppress the weights produced from that initial training.

As explained above in Section 4.1.4, we encountered issues with the trained and validated LIF model once moving to hardware. Between this fragility, and the sometimes chaotic training we saw, we decided to continue our exploration of possible algorithm adjustments despite having obtained models which ultimately did show consistently high performance on the cart-pole task. This work is on-going and will first attempt to test changes to the experience replay buffer size. Though we tested varied buffer sizes against a single model configuration as part of troubleshooting described in Section 4.1.3, we believe it may be beneficial to conduct extended tests against multiple models.

Future work could also include experimentation with using RMSProp as the optimizer instead of AdamW, which is the optimizer used by Mnih et al. [35]. A more substantial change which could be tested in the future would be to attempt alternate training algorithms, such as PPO or a DQN variant such as Double DQN.

4.3.3 Implications for Hardware

One limitation of our software-to-hardware training workflow is in the usage of an existing library, snnTorch, which has been optimized for execution on a GPU, with

vectorized data leveraging a Single Instruction, Multiple Thread (SIMT) architecture. Although this is beneficial for users who wish to run their SNN models on off-the-shelf hardware, the snnTorch code has intentionally not been developed for execution on custom hardware. Some snnTorch architectural decisions which provided constraints in HDL development included the choice of using fully-connected linear layers, with fixed connections between neurons, and the related calculations performed by all neurons at every timestep.

Future work may include usage of a custom software library for training, where a full software/hardware co-design process may be used. This may allow for simplification or customization of network connectivity and neuron firing patterns, allowing sparsely-connected networks or reduced computation per cycle.

4.3.4 Successful Neural Network Characteristics

We found that a neural network using fractional-order neurons was capable of solving the cart-pole task with fewer neurons than either the LIF or bitshift models. This model did, however, show less consistency in performance across multi-seed training, indicating that additional neurons may provide improved stability. The bitshift network type produced the largest total network size in terms of number of neurons, with a history length of only four. This points to the possibility that history did not prove to be an advantageous feature in the bitshift neuron, and that the short history length that was included was either neutral or a hindrance to performance. The fractional-order network performed better at a smaller network size, with a history length of eight. Future work may include determining whether the true fractional-order binomial coefficients make the difference over the bitshift approximations. Though we chose a bitshift pattern to intentionally minimize the difference between the two, as described in Section 3.1.4, it may be the case that even small differences compound over time.

4.4 Summary

In this chapter, we've laid the groundwork for future research into neurons with novel characteristics, such as intrinsic memory, by presenting a complete software-to-hardware workflow using existing open-source software. We've identified a biologically plausible feature as a candidate for digital implementation, and shown that neural networks with fractional-order dynamics may allow for up to a 37.5% reduction in network size while achieving parity with an LIF network in performance on the cart-pole task. In addition, we've established that the particulars of fractional-order dynamics may not be simply replicated with a basic, hardware-efficient bitshift operation, which provides a challenge to future researchers but may also serve as motivation in development of fractional-order analog components.

A primary constraint faced while developing models was the compute time necessary for running Optuna studies and training models. While our results may reflect these challenges, we have provided steps for future work which may readily extend the existing work we performed and presented in this chapter. Making minor algorithm or hyperparameter modifications and running tests are straightforward next steps from the framework we have laid through this work, and may be valuable in further investigations of these neuron models or additional novel neurons.

Chapter 5

Benchmarking SNNs on FPGA

Realizing neural network execution in hardware is key for ensuring applicability to real-world problems. It is also essential for assessing whether any nominal advantages between network types seen during software simulation and training hold when translated to a digital hardware context. To this end, we've synthesized our neural network designs from Chapter 4 for execution on the Nexys A7 FPGA board, comparing networks utilizing our LIF, fractional-order, and bitshift neuron types. We chose the Nexys A7 due to its specific features such as number of DSP slices and onboard peripherals. To execute the cart-pole task on an FPGA, we extended our earlier designs by adding a UART interface. This allowed for communication between the cart-pole environment in software and the SNN performing inference in hardware. In the process, we had to make a number of changes to our original implementation in order to provide a design which could be efficiently synthesized onto hardware. These changes, and the process of synthesizing and executing cart-pole inference on an FPGA are outlined below.

We compare the power, performance, and area characteristics of our LIF, fractional-order, and bitshift networks, noting the tradeoffs between network size and for the fractional-order and bitshift networks with intrinsic memory, the history lengths. In addition, we compare to reported performance for SNN on FPGA implementations in existing work, and against profiles with varying characteristics for our own design.

5.1 Methodology

We broke down the process of creating a communication loop for the cart-pole environment to perform inference on our FPGA into several stages prior to beginning development. This was done in order to validate functionality at each phase prior to moving on to the next. At a high level, the work necessitated creating a communication channel and standard format for messages between the host and FPGA, implementing Python classes to serve as an interface between the Gymnasium [10] environment and the neural network, and validating performance against expected output for a given model. We assessed our implementation by comparing a baseline against a number of different profiles with varied levels of parallelization, and in the case of the fractional-order and bitshift networks with an explicit one-hot FSM encoding for the neurons, using reverse case semantics.

5.1.1 Software-to-Hardware Communication

We began this stage of the project with quantized models which had been validated using software and hardware simulation, and shown to meet our success threshold for the cart-pole task by completing an average of at least 475 steps over the last 100 episodes. Our goal for hardware execution was to be able to run the same cart-pole environment using Gymnasium [10] in software and perform inference for each cart-pole action decision in hardware. This meant that the software would send individual environment observations, consisting of the four state variables for the cart and pole described in Section 2.1.4, and receive back from the hardware an action representing the decision of whether to move the cart right or left.

In moving from hardware simulation to hardware synthesis, we first had to consider the communication protocol between software and hardware. We selected the UART protocol for a number of reasons. One reason was that the FPGA board we were

using included a UART to Universal Serial Bus (USB) bridge. Additionally, we felt that a simpler communication protocol at this stage, as compared to something like Serial Peripheral Interface (SPI), would not be an impediment to accurately estimating inference latency, as we could determine the number of clock cycles required for inference along with the clock frequency used by each type of neural network. Figure 5.1 shows the transaction sequence between the cart-pole environment host running in software and the neural network on the FPGA.

We created cocotb [9] tests to verify basic functionality of the UART modules, including byte fidelity on transmit and receive actions, as well as expected register updates and responses from actions such as a ping, read, write, exec, and edge cases resulting in a bad checksum or other error. To support direct software communication to the FPGA over UART, we created an FPGA interface class within Python. This contained all the logic for building a frame to send to the FPGA, along with error handling and helper functions. This Python class was used to verify basic communication with the board, and was implemented such that it could be leveraged by a script which would perform the cart-pole evaluation on the hardware.

5.1.2 Inference Validation

With the communication protocol implemented and verified, we created a top-level module for the board in order to validate communication with a neural network over UART. The goal at this stage was to be able to perform one inference step and verify that the FPGA returned data matching our expectations. This entailed creating golden vectors of expected results for a set of inputs provided to a given model. To produce these expected results, we implemented fixed-point versions of our neural networks in software, which matched our hardware implementations. These result vectors were independent of an evaluation trajectory where the goal was for the model to keep the pole upright, and simply validated that a set of inputs produced bit-exact

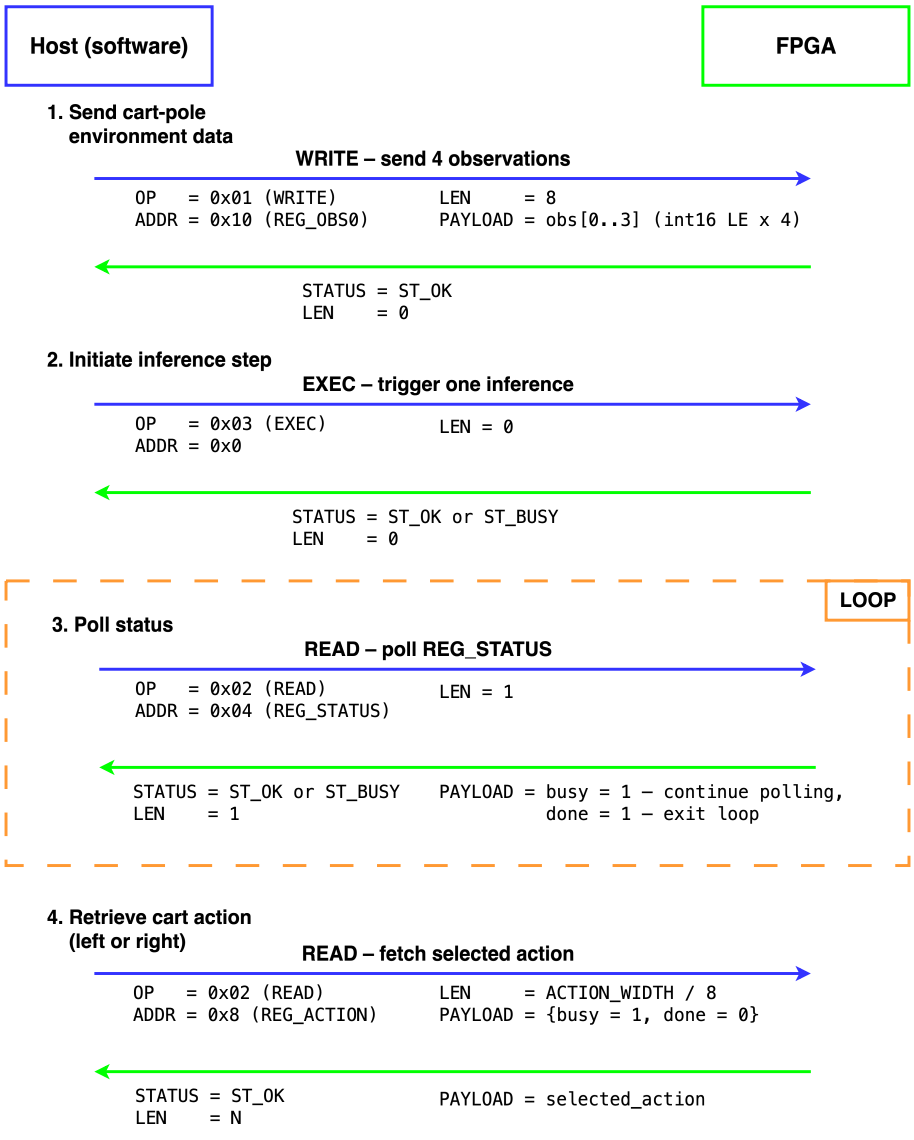

Figure 5.1: Transaction sequence between the cart-pole environment host in software and the FPGA running the trained neural network. The host begins by sending the four environment observations and initiating an inference on the FPGA, then waits while the FPGA is busy. When the FPGA asserts the `done` signal, the host reads the selected action for moving the cart left or right.

outputs.

Figure 5.2 shows a schematic for the top-level module and board-level I/O, with UART receive and transmit signals, as well as heartbeat signals to validate board bring-up and data transmission. The environment observations are transferred over the `uart_rx_i` signal. Figure 5.3 shows the neural network schematic, where the UART wrapper for the hardware accelerator instantiates a neural network instance, which

receives the observations and returns the resulting action. The top-level timing with handshaking signals is shown in Figure 5.4, with inference latency being dependent on neural network type, network size, and level parallelization utilized.

The top module and `neural_network` modules were parameterized such that the neuron model type, hidden layer sizes, history length for fractional-order and bitshift neurons, number of timesteps per inference step, and bit widths of several data types such as the binomial coefficients and membrane potential width were all configurable, with reasonable defaults. At this point, we also updated the constraints file to set the desired clock speed and initialize the onboard UART transmit and receive channels.

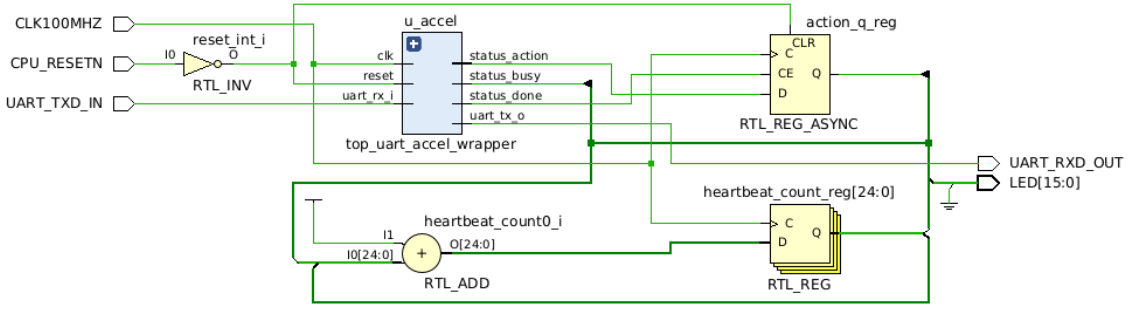

Figure 5.2: Vivado schematic showing board-level signals used as a part of board bring-up and initial validation, including a heartbeat signal displayed using the on-board LEDs. Environment observations are transferred to the top-level module through the `uart_rx_i` signal, which then passes the data to the neural network.

With our functionally equivalent fixed-point software neural networks, we created golden vector files with the expected output data for a given sequence of environment observations, producing one golden vector file for each model we tested. The output data included both the expected chosen action as well as two Q-values from a `q_accumulator` SV module, where one value was associated with each possible output (moving the cart right or left), and the greater of the two determined the selected action. After a ping test to confirm communication with the board had been established, we ran a validation script testing one inference step at a time, comparing the golden vector data to the output provided by the neural network running on the

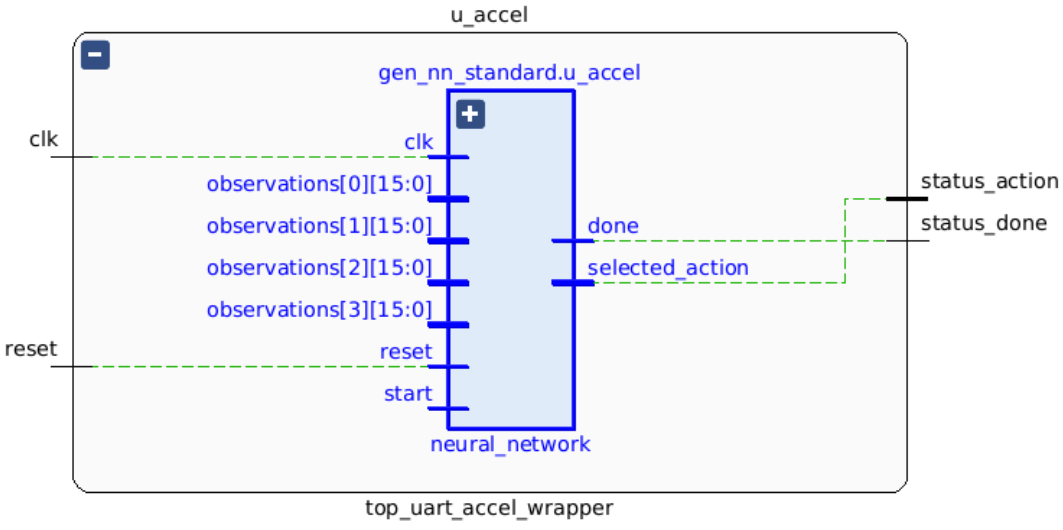

Figure 5.3: Top-level schematic from Vivado showing the UART hardware accelerator wrapper interface. The top module instantiates a neural network, which receives the environment observations and returns a selected action. Control signals such as `reset`, `start`, and `done` are used for handshaking between software and hardware.

board. This did not yet include usage of the Gymnasium library [47] and its associated cart-pole environment, but was rather a simple input to output functional check.

5.1.3 Cart-Pole Execution

To perform end-to-end execution of the cart-pole task using our neural networks running on the FPGA, we created a class for our SNN which could be called by a script running the Gymnasium cart-pole environment [10]. Rather than calculating the output directly in software as seen in Listing 4.1, this version sent the environment observations to the FPGA board and awaited the response, which it then returned to the calling function in the environment. Listing 5.1 shows the implementation of the `forward` method which is used to perform inference in hardware on the FPGA board. It is the equivalent of the software inference seen in Listing 4.1.

In this code, the SNN uses an instance of an `FpgaInterface` class which handles communication with the board over UART. This `self._fpga` instance has methods for sending the environment observation values (`write_obs`) and triggering an inference

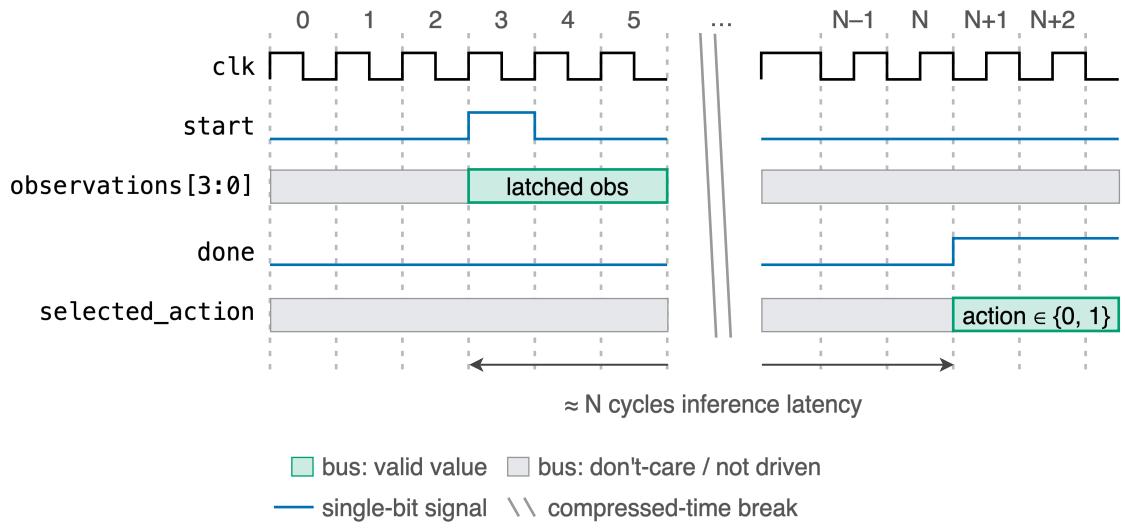

Figure 5.4: Top-level timing diagram showing single inference step. The control signals `start` and `done` allow coordination between the software and hardware. The number of cycles for inference is variable depending on the neural network type, network size, and parallelization configuration.

and waiting for the result (`exec_and_read_action`).

Using the weights from our models trained in Chapter 4, we were able to run the cart-pole task inference on the FPGA using this implementation. We confirmed performance from hardware simulation was maintained while executing on the FPGA, with at least an average reward of 475 over the last 100 episodes of a cart-pole evaluation-only run.

5.1.4 SV Modifications

One modification we made after initial attempts at synthesis and subsequent evaluation of our models was to drop the constant C, as seen in Equation 2.9. With a Δt value of 1.0, which was what we used throughout training, the C constant would always be 1.0. With the inclusion of this term, the maximum bit widths of intermediate terms in our membrane potential calculations for the fractional-order and bitshift neurons reached up to 80 bits wide due to the scaling and overflow handling required for inclusion of this 16-bit value at our default bit widths and fixed-point formats of Q2.13 for input `current`, Q10.13 for membrane potential in the output and internal

```
# Convert observations to a plain numpy array
obs_np = observations.squeeze(0).detach().cpu().numpy()
# Convert values in numpy array to fixed point for hardware
obs_q = float_to_qs2_13(obs_np)

# Send environment observations to FPGA
self._fpga.write_obs(obs_q)
# Trigger inference and wait for result
action = self._fpga.exec_and_read_action()

# One-hot the FPGA's chosen action so callers can easily extract action
result = torch.zeros(1, self.n_actions)
result[0, action] = 1.0
return result
```

Listing 5.1: Inference step performed within the hardware wrapper class's `forward` method, with cart-pole environment `observations` as input and the output `result` representing the selected action for moving the cart left or right. The `self._fpga` attribute here is an instance of an `FpgaInterface` class which we created to facilitate communication between software and hardware. This snippet replaces the equivalent from Listing 4.1, where the inference is performed in software.

representations, and necessary intermediate terms. The multiplication of the C value times the *history_term* as seen in Equation 2.9 was particularly expensive, due to fixed-point requirements for determining bit widths of result terms where bits for the integer and fractional portions are additive. With this change, the maximum bit width of the terms used in the calculation was reduced to 62 bits. We did lose the option to test alternative values of Δt in this iteration, however, our existing models utilized only $\Delta t = 1.0$. Future work could include reintroducing the C constant and testing varying Δt values, which would be an additional lever for adjusting the fractional-order dynamics.

Another adjustment we made was to the `bitshift_lif` module. The resulting network configuration from Chapter 4 was the largest in terms of network size, at hidden layer sizes of 64 and 32 for layers one and two, respectively. Initial attempts to synthesize this module failed, with an over-utilization of LUTs. On investigation into the synthesis utilization report, we determined that instead of inferring DSP

slices as we had intended, the multiply followed by a shift operation used to apply the λ leakage term seen in Equation 2.9 was being put into LUTs instead. This was likely due to the bit width of the operands, which would have resulted in Vivado using two DSP slices per operation, and was therefore deemed too inefficient by Vivado optimization. We decided to break up this sequence of operations across two different states in our FSM, so that we could ensure utilization of DSP slices instead of LUTs. Making this change resulted in completion of synthesis and implementation for the bitshift neural network with successful timing closure.

5.1.5 SV Configuration Profiles

Once we had all of our core models synthesized and validated in cart-pole inference performance on the FPGA, we created alternate synthesis profiles for exploration of power, performance, and area characteristics under different conditions impacting synthesis output but not the core logic. This primarily focused on varying degrees of parallelism, but also included a profile for explicit one-hot encoding of the FSMs used in the fractional-order and bitshift neurons, with reverse `case` statement semantics. The LIF neuron did not require an FSM and was thus not included in this profile. In the baseline configuration, Vivado was left to automatically determine the FSM encoding to be used.

Our parallelism adjustments centered on the parameterized batch processing available in our `linear_layer` and `q_accumulator` modules. Figure 5.5 shows an example of the primary calculation performed in the `linear_layer` module where inputs are multiplied by the layer's weights. In this case with full parallelism, each input is multiplied by a weight vector equal in length to the number of layer outputs each cycle. Figure 5.6 shows an example where the module's parameters are such that the parallelism is reduced as compared to the maximum possible, where each input is multiplied by only a portion of the a given weight vector each cycle. This was necessary

in the case of the LIF and bitshift networks, which had a larger number of neurons than the fractional-order network in each layer. Since the parallelism degree corresponded to DSP utilization in addition to increased routing pressure, the larger networks required reduced parallelism from the default where the parallelism P was equal to the number of outputs of the layer. With this parallelized implementation for the `linear_layer`, the latency for this module follows the formula $1 + \frac{num\_outputs}{parallelism} \times (num\_inputs + 1)$ cycles.

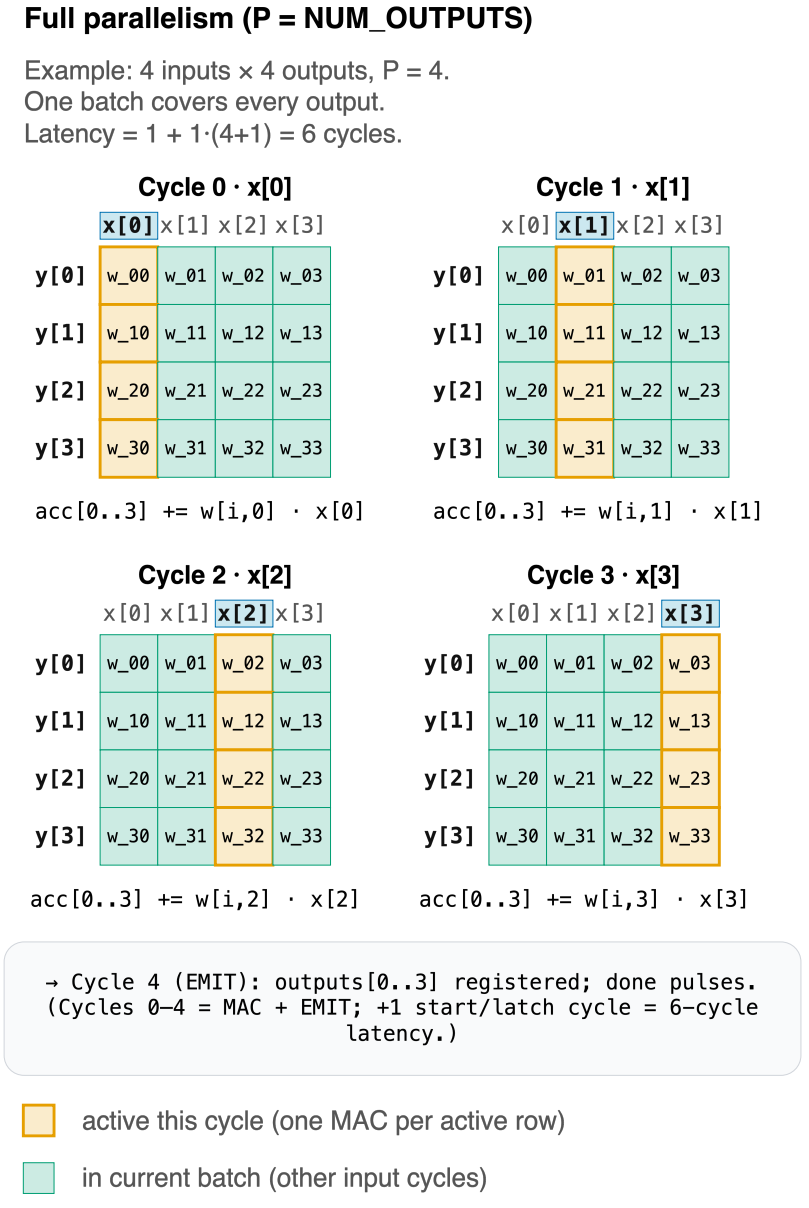

Figure 5.5: Full parallelism example, where parallelism P is equal to the number of outputs for the `linear_layer` instance. P MAC operations perform in lockstep, and each utilizes a dedicated DSP slice. A single input is multiplied by a weight vector of length P.

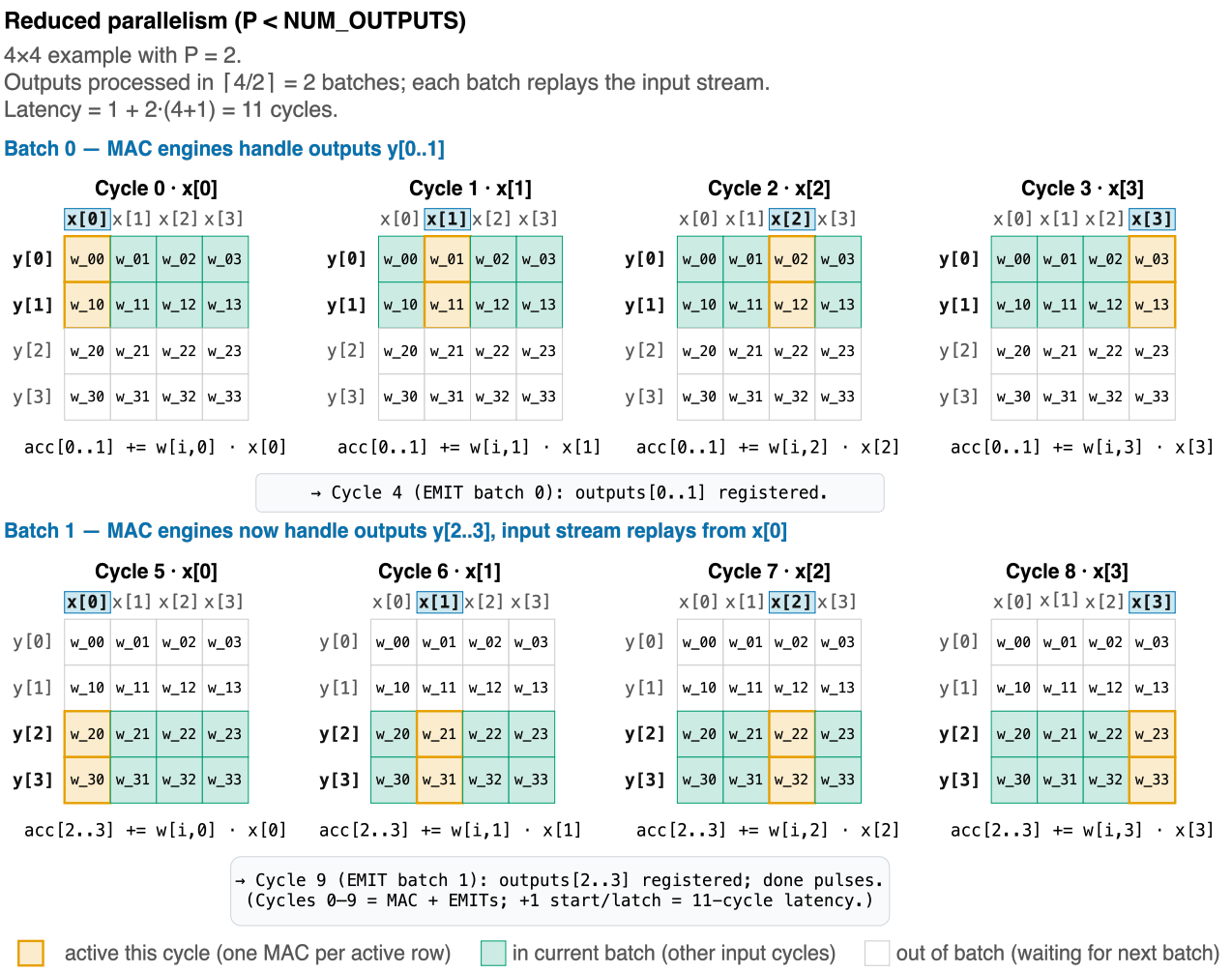

Figure 5.6: Reduced parallelism example, where each input is multiplied by a weight vector of length $\frac{num\_outputs}{P}$. In this case, the number of cycles required to calculate the output is scaled by $\frac{num\_outputs}{parallelism}$, or two in this example, resulting in 11 cycles total when accounting for the initialization and final emit cycles.

Our `q_accumulator` module, which reads the second hidden layer's neuron membrane outputs from a buffer and calculates the resulting Q-values for each possible action, has a similar parameterized parallelization. The latency in this module is calculated as $num\_timesteps \times \frac{num\_neurons}{batch\_size+4} + 2$ cycles, where $num\_timesteps$ is the number of timesteps during which the input is held constant as a form of rate-encoding, with the snnTorch [8] default being 30 steps. The value $batch\_size$ in this case refers to the parameterized batch value that the `q_accumulator` uses to determine how many membrane potential values from the buffer to process at once, and $num\_neurons$ refers to the number of neurons in the final hidden layer.

The four reduced parallelism cases we considered included the following:

1. **fc1 batch 8**: The batch size for the first `linear_layer` instance in each network, `fc1`, was set to eight.

2. **fc2 batch 8**: The batch size for the second `linear_layer` instance in each network, `fc2`, was set to eight.

3. **Q batch 1, fc2 batch 8**: The batch size for the `q_accumulator` was set to one, and the batch size for `fc2` was set to eight.

4. **Q batch 1**: The batch size for the `q_accumulator` was set to one.

In the baseline profiles we compared against, both the LIF and bitshift networks had parallelism settings of 16 for `fc1` and `fc2`. The fractional-order network had the default parallelism setting, using the `num_outputs` values associated with each fully-connected linear layer. In the case of our trained network from Chapter 4 with hidden layer sizes of 32 and 8, respectively, this meant that `fc1` had a parallelism setting of 32, and `fc2` a setting of eight. We used a batch size of four for the `q_accumulator` in our baseline profiles for all networks. These settings were determined by the maximum parallelism possible to achieve while meeting timing closure, where we started at

$P = num\_outputs$ for each network and reduced parallelism from there until we met timing closure.

5.2 Results

We collected reported implementation metrics from Vivado for our baseline, reduced parallelism, and one-hot encoding profiles to evaluate resource usage, timing, and estimated power consumption. To determine the number of cycles per inference, we implemented cocotb [9] tests which counted cycles and wrote results to a file that was later used in analysis. Below, we present our comparisons between profiles within our design, as well as high-level comparisons to alternate implementations in the literature.

5.2.1 Resource Usage

Table 5.1 shows resource usage across our baseline configurations. With the largest number of neurons, the bitshift network consumed the most board resources of any of our tested networks. The fractional-order network, despite having a smaller number of total neurons than the LIF, utilized a higher number of resources, including LUTs, registers, and DSP blocks. This is due to the additional logic for calculation and incorporation of the membrane potential history.

Resource	**LIF**	**Fractional-order**	**Bitshift**
Slice LUTs	12 560	17 345	36 733
Slice Registers	21 492	21 511	45 492
DSP Slices	104	208	232

Table 5.1: FPGA resource utilization for the baseline LIF, fractional-order, and bitshift neural networks, obtained from Vivado implementation reports on the Nexys A7 (Xilinx Artix-7 XC7A100T). No BRAM tiles or LUTs as memory were used by any design.

Figure 5.7 shows the percentage of board resources each network utilized. Both designs incorporating intrinsic memory required more DSP slices than the LIF network. For the fractional-order network, this was due to both the membrane potential history

calculation itself, as well as the increased parallelism that the design was able to leverage in its first `linear_layer` instance. For the bitshift network, the large number of DSP slices was due to the increased total network size, with hidden layer sizes of 64 and 32.

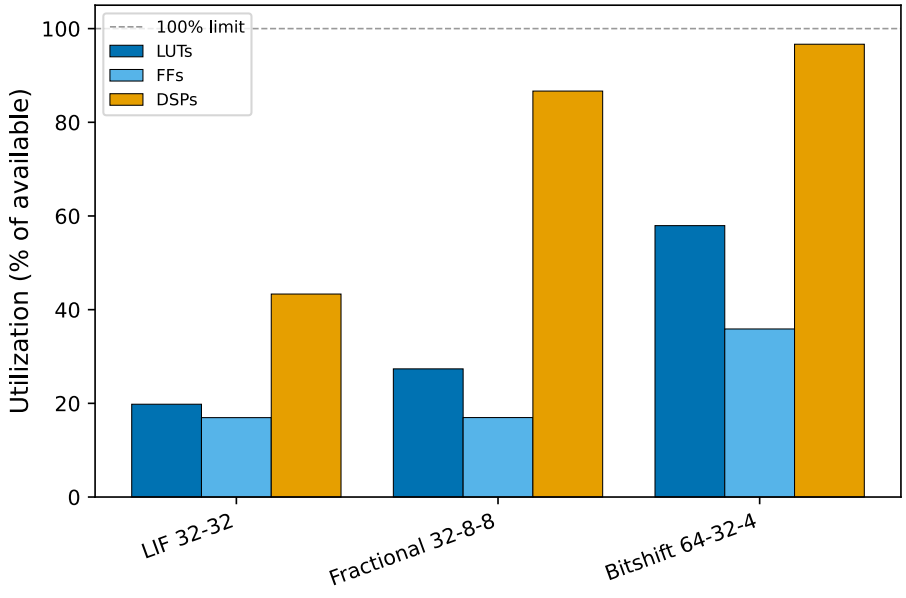

Figure 5.7: Resource utilization across the baseline profiles for the LIF, fractional-order, and bitshift networks. The fractional-order network has hidden layer one and two sizes of 32 and eight, respectively, with a history length of eight. The bitshift has hidden layer sizes of 64 and 32, with a history length of four, while the LIF has hidden layer sizes of 32 each. The bitshift network used nearly all of the available DSP slices, due to the size of the network and the utilization of this resource within the fully-connected linear layer.

Figure 5.8 shows the change in resource usage from the baseline profile for each network across the five additional profiles we tested. Resource usage declined with reduced parallelism as expected, except in the case where `fc2` in the fractional-order network was set to eight, which was the same as the default derived from the hidden layer two value of the same size (eight). This slight increase was likely due to minor changes resulting from Vivado's optimization of the design during implementation, which also accounts for the reduced LUTs. The LIF network saw larger reductions in resource usage as a percentage than the bitshift network, with the same parallelism adjustments. This reflects the overall higher resource consumption of the bitshift

network as compared to the LIF, with utilization of resources ranging from ≈111-192% higher. Similarly, the fractional-order saw generally smaller reductions in resource usage, with the exception of the reduced parallelism in `fc1`. Unlike the bitshift network, these differences in resource usage came from the total network size being smaller to begin with, so that the reduction in parallelism in `fc2`, as noted above, was nominal. The reduction in parallelism for `fc1` for the fractional-order network had the largest impact, due to the reduction in parallelism from 32 to eight. As seen in Figure 5.6, this increased the number of cycles required to compute the output values by a factor of $\frac{num\_outputs}{parallelism}$, or four in this case. Table 5.3 summarizes relative resource usage of the three networks.

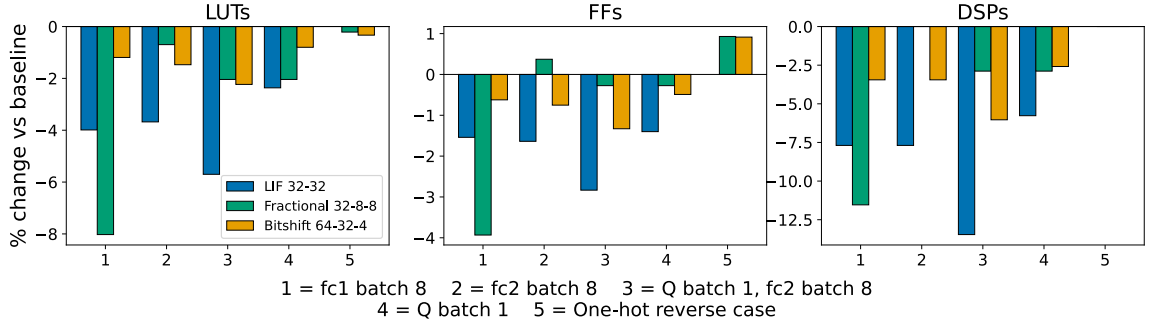

Figure 5.8: Percent change in resource usage as compared to the baseline profile for profiles with reduced parallelism in the fully-connected linear layer one (fc1 batch 8), fully-connected linear layer two (fc2 batch 8), both the Q accumulator and fully-connected linear layer 2 (Q batch 1, fc2 batch 8), only the Q accumulator (Q batch 1), and a profile where parallelization was maintained, but Vivado's automatic FSM encoding for the fractional and bitshift neurons was overridden to explicitly use one-hot encoding with reverse case statement semantics. The greatest impact on fractional-order resource usage came from reducing parallelism in `fc1`, while the bitshift and LIF networks saw varying ranges of resource reduction with lowered parallelism.

For the fractional-order and bitshift networks, the explicit one-hot encoding with reverse case semantics resulted in an increase in flip-flop utilization and slight decrease in LUT utilization. This is an expected change, as each state requires more flip-flops to encode the state with one-hot encoding, but less complex decode logic. DSP usage is unaffected by these changes.

While Figure 5.7 shows that none of our networks were severely constrained in their

usage of LUTs or flip-flops on this board, our alternate profiles show that reductions in area are possible, which may be desirable for more resource-constrained environments. This comes at the cost of increased latency, which is discussed below in Section 5.2.3.

5.2.2 Power Usage

Figure 5.9 shows Vivado's estimates for static and dynamic power for our different networks and their profiles. Reductions in parallelism typically resulted in either minor reductions in dynamic power or a comparable power estimate to the baseline. The notable exception is the profile where one-hot encoding with reverse case semantics were implemented in the fractional-order and bitshift networks. This is in line with expectations given the increased flip-flop utilization associated with one-hot encoding.

For our baseline profiles, the fractional-order network had ≈64% higher estimated dynamic power usage over the LIF, with 0.427 W for the fractional-order versus 0.260 W for the LIF. The bitshift network had ≈94% higher estimated dynamic power as compared to the LIF, at 0.505 W.

Table 5.2 shows a comparison of power usage and maximum clock frequency between our baseline implementations and several examples from the literature. These examples were chosen because they used networks composed of LIF neuron models or a similar variant. Additionally, the FPGAs used were all Xilinx and several were in the same device family as our board, the Nexys A7 with the Artix-7. This standardization lends improved ability to compare between implementations and metric reporting, though we note here that these implementations from the literature vary in their target task. Our networks largely fell within the range of power usage seen in prior work with the same or similar maximum clock frequency.

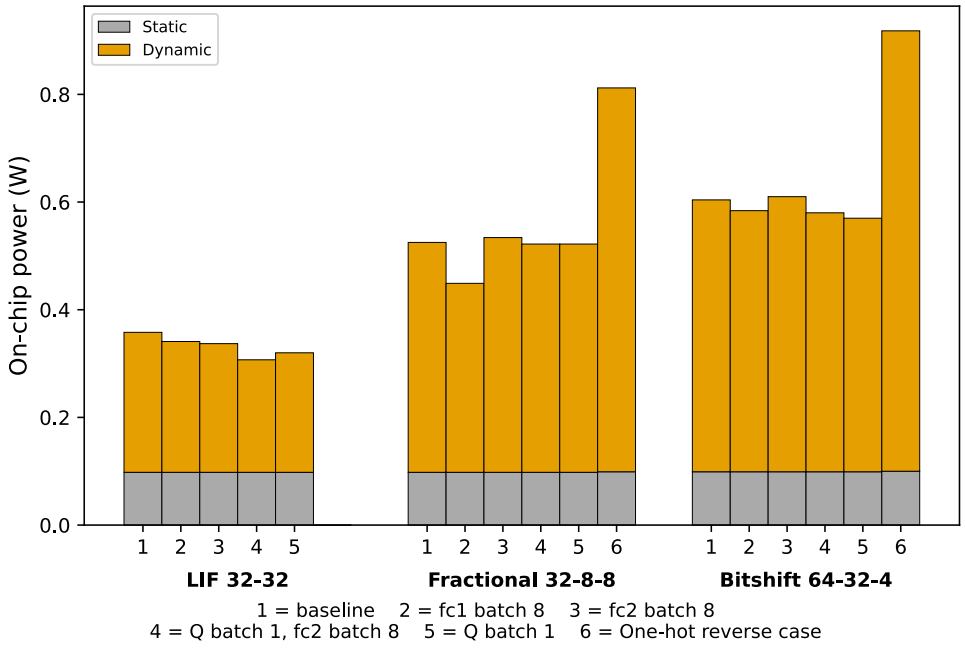

Figure 5.9: Estimates for static and dynamic power for all network types and profiles. Dynamic power was typically slightly reduced or comparable to the baseline in the reduced parallelism profiles. A notable increase in dynamic power was seen in the fractional-order and bitshift profiles where FSM encoding was explicitly configured to be one-hot, with reverse case semantics. For the fractional-order network, dynamic power increased by 67% in the one-hot profile, while the bitshift saw a 62% increase.

Work	$F_{\max}$ **(MHz)**	**Power (W)**	**FPGA**
Ali et al. [51]	67	0.495	Artix-7
Carpegna et al. [28]	100	0.180 to 0.430	Zynq UltraScale+
Mishra et al. [52]	100	0.426	Artix-7
Zhang et al. [53]	233*	1.540	Zynq UltraScale+
This work (LIF)	100.05	0.358	Artix-7 (XC7A100T)
This work (fractional-order)	101.44	0.525	Artix-7 (XC7A100T)
This work (bitshift)	102.67	0.603	Artix-7 (XC7A100T)

Table 5.2: Comparison of maximum clock frequency and total power against prior FPGA SNN implementations using LIF variants. All designs target Xilinx FPGAs. Frequency and power values for this work are post-implementation estimates from Vivado on the Nexys A7. *Zhang et al. report a 233 MHz maximum synthesized frequency but evaluate at 200 MHz. Carpegna et al. report power as a range across configurations targeting different tasks.

5.2.3 Performance

Figure 5.10 shows the total cycles per inference step. The fractional-order network showed the smallest change across profiles, as the calculation in the `fc2` linear layer

is the largest contributor to number of cycles, and this network was already at a size of eight in the baseline profile for this layer. In addition, due to the small `fc2` size in comparison to the LIF and bitshift networks, the fractional-order network required the smallest number of cycles per inference total. For the baseline profiles, the fractional-order network had ≈31% lower latency than the LIF, with 592 cycles versus 857, per inference step. The bitshift network had a ≈272% higher latency, with 3187 cycles per inference step.

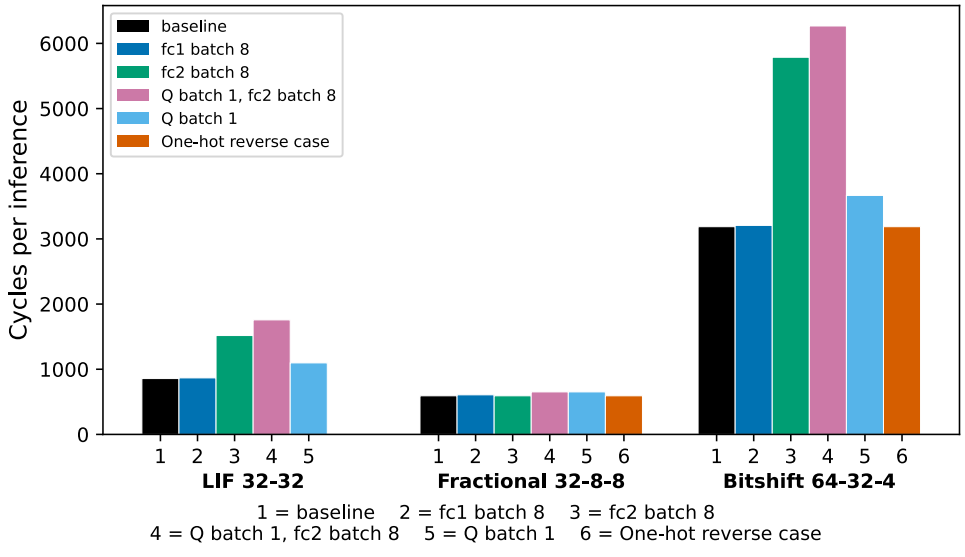

Figure 5.10: Cycles per inference across network types and profiles. The fractional-order network required the smallest number of cycles per inference total due to the small `fc2` size in comparison to the LIF and bitshift network. Relatedly, this network showed the smallest impact from adjustments to parallelism. Both the LIF and bitshift networks saw larger relative increases in number of cycles per inference from reducing parallelism in `fc2`, with a 77% increase for the LIF and 81.6% increase for the bitshift between the baseline and "fc2 batch 8" profile.

We compared maximum clock frequencies across profiles, as reported by Vivado, and found that all profiles for the fractional-order and bitshift networks ranged from ≈100.5 MHz to 102.6 MHz. The explicit one-hot encoding with reverse case semantics was at the lower end among the profiles. For the LIF network, the maximum clock frequencies ranged from ≈99.3 MHz to 100.05 MHz, indicating that reduced parallelism resulted in minor performance degradation.

Table 5.2 shows the maximum clock frequency capable within each network's

baseline profile as compared to clock frequency and power for comparable work from the literature. Our designs achieved maximum clock frequencies of ≈100 MHz, which is in line with targets used in prior work, particularly within a similar class of power consumption.

Table 5.3 summarizes the relative cost of the fractional-order and bitshift networks versus the LIF network, using data from the baseline profile for each, reporting power, performance in terms of latency, and area.

Metric (vs. LIF)	**Fractional-order**	**Bitshift**
Dynamic power	+64.2%	+94.2%
Latency (cycles)	−30.9%	+271.9%
Slice LUTs	+38.1%	+192.5%
Flip-flops	+0.1%	+111.7%
DSP blocks	+100.0%	+123.1%

Table 5.3: Relative cost of the fractional-order and bitshift networks against the LIF network, given as percent change from LIF. Power is dynamic power; latency is total clock cycles per inference; area is reported as LUTs, flip-flops, and DSP slices. All values are from the baseline profile using Vivado post-implementation results. Static power is omitted, as it is nearly constant across the three networks (≈ 0.098–0.099 W).

5.3 Discussion

Based on our analysis and comparison of networks using the three different neuron models LIF, fractional-order, and bitshift, the total network size appears to be the dominant factor in determining power, performance, and area outcomes. In particular, the size of hidden layer two, with the preceding `linear_layer` instance `fc2` in our networks, had the largest impact on these metrics. This is due to the second linear layer's connectivity, where its input size is equal to the first hidden layer's size and its output size is equal to the second hidden layer's size. The bitshift network was the most severely impacted by this, with its hidden layer sizes of 64 and 32, respectively. Future work could include creating a separate `linear_layer` module for the second

layer, leaving the current functionality in place for the first linear layer. Though the connectivity of the second linear layer is dictated by the structure in this fully-connected, feed-forward network, a dedicated SV module could be optimized for the second layer by taking better advantage of spike sparsity in its input spike vector, reducing the usage of DSP slices.

In the case of our model configurations resulting from work in Chapter 4, the history lengths for our networks with intrinsic memory were relatively short, at a length of eight for the fractional-order network and four for the bitshift network. The values of four and eight were the two minimum possible values in the Optuna search space, with the maximum possible being 128, and values of 16, 32, and 64 in between. The contribution of the history length to the latency in each timestep is $history\_length + 5$ for the fractional-order network and $history\_length + 6$ for the bitshift, while the latency in the linear layers, as first described in Section 5.1.5, is $1 + \frac{num\_outputs}{parallelism} \times (num\_inputs + 1)$ cycles. In the case of the fractional-order network, this translated to a cost of 34 cycles for the second linear layer's calculation using full parallelism, versus 13 for the history calculation. For the bitshift network, the second linear layer cost was 131 cycles with reduced parallelism, and the history calculation was 10 cycles. With these configurations, the network size dominated over the history length in impact on latency. However, as shown in Table 5.1, despite having a smaller total network size than the LIF network, the fractional-order network consumed more resources. This increased area contributed to increased dynamic power, as shown in Figure 5.9.

Table 5.4 shows the calculated latency and energy used per cart-pole inference, based on the estimated power, measured number of cycles, and maximum clock frequency. Though the fractional-order network completes inference in fewer cycles and has the lowest latency at 5.84 μs as compared to 8.57 μs for the LIF network, their total energy consumption per inference is nearly identical at 3.06 μJ for the

fractional-order and 3.07 μJ for the LIF. The bitshift network was inferior in nearly all respects due to its larger network size.

Network	**Power (W)**	**Cycles**	$F_{\max}$ **(MHz)**	**Latency (μs)**	**Energy (μJ)**
LIF	0.358	857	100.05	8.57	3.07
Fractional-order	0.525	592	101.44	5.84	3.06
Bitshift	0.603	3187	102.67	31.04	18.72

Table 5.4: Per-inference latency and energy for the three neural networks, derived from Vivado post-implementation power and frequency estimates. Latency is computed as cycles/$F_{\max}$ and energy as $Power \times$ latency. The LIF and fractional-order designs are effectively tied near 3.07 μJ, while the bitshift design consumes $\approx 6\times$ more energy, driven by its larger network size and resulting higher cycle count.

Our investigation into alternate profiles showed reducing parallelism could be an effective means to decrease resource usage, with varying reductions across LUTs, flip-flops, and DSP slices shown in Figure 5.8. However, power usage typically did not decrease, and in the case of reduced parallelism in the second linear layer (`fc2`), latency increased ≈77% for the LIF network and ≈81.6% for the bitshift network. This indicates that reducing parallelism may only be beneficial in the case of severely area-constrained devices, if increased latency and energy usage per inference is an acceptable tradeoff. For the fractional-order and bitshift one-hot FSM encoding profiles, we did not see any maximum clock frequency improvement, indicating that allowing Vivado to select default FSM encoding may be the best option. Additional research could involve testing one-hot FSM encoding with reverse case semantics in other FSMs used throughout the design, including the FSM in the `neural_network` module. Though the neuron-level encoding is impactful as the network includes many neuron instances versus a single `neural_network` instance, it may be the case that a larger FSM shows a different performance response to changed encoding.

5.4 Summary

This chapter covered the required additions necessary to extend our cart-pole benchmarking on three different neural networks from hardware simulation to execution on synthesized hardware. In the process of meeting timing closure for all of our network configurations, we made adjustments such as breaking up long combinational paths with registering or extending the number of FSM states, and leveraging parameterized parallelism to reduce resource usage, though typically at the cost of latency. In the configurations resulting from Chapter 4, the dominant performance factor proved to be the total network size rather than the history length.

For the network configurations tested here, we found that the total size of the bitshift network impacted the resulting power, performance, and area characteristics to such a degree that it was not competitive with the LIF or fractional-order networks. The fractional-order network was superior to the LIF in terms of latency per cart-pole inference, though the energy usage between the two was nearly identical. These findings indicate that the reduced size of fractional-order networks as compared to LIF network imparts some benefits, though the determination as to which network type to use depends on the particular environment constraints related to latency, energy usage, and FPGA resource utilization.

Chapter 6

Conclusion & Future Work

6.1 Summary

In this work, we investigated the impact of intrinsic memory of membrane potential values within neuron models used in SNNs. We implemented two novel neuron models which incorporated intrinsic memory, one of which used a fractional-order calculation based on the GL formulation and the other being an approximation of fractional-order dynamics using bitshift operations in place of multiplication. These two models were compared against a baseline LIF neuron, a standard implementation commonly used in SNNs. Our specific contributions are listed above in Section 1.3.

Chapter 2 introduced SNNs and the LIF neuron, fractional-order calculus, the benchmarking cart-pole task, DQN for training, as well as the mathematical foundations for our neuron models. We provided examples of fractional-order update equation calculations using the GL approximation, upon which our software and hardware models were based.

In Chapter 3, we presented implementations of our three neuron models in HDL. We analyzed the requirements for inclusion of intrinsic memory in both the fractional-order and bitshift neurons, and provided a trade-off analysis where we quantified the limits of history inclusion based on digital representation of key operands used for incorporation of intrinsic memory, such as binomial coefficients and fractional-order α value. We compared possible bitshift sequences against the binomial coefficients calculated for $\alpha = 0.5$ and proposed a "custom slow decay" sequence which provided

a maximum history length of 105 and reduced relative error as compared to three simpler alternatives.

In Chapter 4, we outlined a software-to-hardware workflow for development and testing of novel neuron models using open-source software. Using this workflow, we trained models on the cart-pole task, quantized the models for hardware execution, validated the resulting quantized models in software, and finished by validating performance in hardware simulation with a complete, end-to-end cart-pole integration test. This was in preparation for the final piece of our work, in which we used those validated models for inference in the cart-pole task on hardware.

Chapter 5 provided details on the necessary additions and modifications we made to support execution of our neural networks on the Nexys A7 FPGA board with the Artix-7 (XC7A100T) chip. We compared baseline profiles for each network type, including the top-performing LIF, fractional-order, and bitshift models from Chapter 4, between network type as well as against alternate synthesis profiles with different parallelization or FSM encoding characteristics within each network type. We also compared against prior work, finding that our maximum clock frequency and power usage was similar to comparable work testing LIF-based SNNs on FPGAs.

In summary, we found that the bitshift model resulted in a larger neural network in terms of number of neurons in our training. It did not produce any advantages over the LIF in terms of inference accuracy or power, performance, and area characteristics in hardware. Our fractional-order model did result in a smaller neural network than the LIF, with a 37.5% reduction in size. This translated to ≈31% lower latency per inference step in the cart-pole task. However, the additional board resources such as LUTs, flip-flops, and DSP slices for the fractional-order network contributed to an total energy usage per inference step of 3.06 μJ, nearly identical to the energy usage of 3.07 μJ for the LIF network.

6.2 Future Work

In future work, we hope to revisit and fine-tune our training algorithm in order to achieve better stabilization across Optuna trials as well as multi-seed retraining of candidate model configurations. Though we saw evidence of stabilization, particularly at larger network sizes, we could gain greater confidence in our models by reducing instances of catastrophic forgetting or chaotic training patterns. Specifically, we would plan to continue investigation into hyperparameter adjustments such as the experience replay buffer size for the current DQN algorithm, and consider whether a PPO algorithm would be worth testing. With these adjustments, we could then run extended Optuna studies beyond the $\approx$100 trials per study we were able to run in this work.

For the HDL, future work may include additional optimizations for the second linear layer in the network, which proved to have a large impact on the synthesis results given its high degree of connectivity in our fully-connected, feed-forward networks. We would consider a dedicated SV module for this layer, distinct from the first linear layer. Opportunities for improved performance may come from leveraging the potential sparsity of the input spike vectors.

More broadly, to carry this work forward, we would like to target an alternative software implementation for translation to hardware. Though there are benefits to using an open-source software library such as snnTorch [8], it is a general-purpose tool and as such, it is not possible to engage in a complete hardware/software co-design process using it. Recent work from Ge et al. [54] includes a PyTorch-compatible fractional-order SNN library, which is one possible option for a starting point. With a custom-built or dedicated software library to begin with, we may have greater flexibility in adjusting connectivity, for example, reducing routing and computation pressure on the internal layers of the network.

The author declares that NotebookLM generative AI was used for assistance in data compilation across reviewed papers within Tables 2.1 and 2.2, with all data manually verified. ChatGPT was used for assistance in generating LaTeX-formatted content and producing the examples in Section 2.2.3 and Section 2.2.4, with all content and calculations manually verified. Figures 4.3, 5.4, 5.5, 5.6 were created with initial drafts produced by Claude, which were revised and finalized manually by the author using draw.io. Multiple LLMs were used for assistance in code generation after initial manual planning and design. LLM code generation was guided by the specifications and requirements provided by the author, and the final code was reviewed and edited by the author to ensure correctness and adherence to the intended functionality.

Bibliography

[1] International Energy Agency, "Energy and AI," International Energy Agency, Paris, Tech. Rep., 2025, licence: CC BY 4.0. Accessed January 24, 2026. [Online]. Available: https://www.iea.org/reports/energy-and-ai

[2] S. Abreu, S. B. Shrestha, R.-J. Zhu, and J. Eshraghian, "Neuromorphic principles for efficient large language models on Intel Loihi 2," in *ICLR 2025 Workshop (SCOPE)*, 2025, arXiv:2503.18002. [Online]. Available: https://arxiv.org/abs/2503.18002

[3] D. Kudithipudi, C. Schuman, C. M. Vineyard, T. Pandit, C. Merkel, R. Kubendran, J. B. Aimone, G. Orchard, C. Mayr, R. Benosman, J. Hays, C. Young, C. Bartolozzi, A. Majumdar, S. G. Cardwell, M. Payvand, S. Buckley, S. Kulkarni, H. A. Gonzalez, G. Cauwenberghs, C. S. Thakur, A. Subramoney, and S. Furber, "Neuromorphic computing at scale," *Nature*, vol. 637, no. 8047, pp. 801–812, 2025.

[4] W. Teka, T. M. Marinov, and F. Santamaria, "Neuronal Spike Timing Adaptation Described with a Fractional Leaky Integrate-and-Fire Model," *PLOS Computational Biology*, vol. 10, no. 3, p. e1003526, 2014.

[5] B. N. Lundstrom, M. H. Higgs, W. J. Spain, and A. L. Fairhall, "Fractional differentiation by neocortical pyramidal neurons," *Nature neuroscience*, vol. 11, no. 11, pp. 1335–1342, 2008.

[6] D. Clemente-López, J. M. Munoz-Pacheco, and J. D. J. Rangel-Magdaleno, "A Review of the Digital Implementation of Continuous-Time Fractional-Order Chaotic

Systems Using FPGAs and Embedded Hardware," *Archives of Computational Methods in Engineering*, vol. 30, no. 2, pp. 951–983, 2023.

[7] K.-U. Demasius, A. Kirschen, and S. Parkin, "Energy-efficient memcapacitor devices for neuromorphic computing," *Nature Electronics*, vol. 4, no. 10, pp. 748–756, 2021.

[8] J. Eshraghian, "snnTorch," [Online] https://github.com/jeshraghian/snntorch, 2023, [Accessed: Jul. 15, 2025].

[9] "cocotb: A coroutine-based cosimulation testbench library," [Online] Available: https://www.cocotb.org/, 2025, [Accessed: Jul. 15, 2025]. [Online]. Available: https://github.com/cocotb/cocotb

[10] "Gymnasium Documentation," [Online] Available https://gymnasium.farama.org/index.html, 2026, [Accessed: Oct. 10, 2025]. [Online]. Available: https://gymnasium.farama.org/index.html

[11] J. S. Plank, C. P. Rizzo, C. A. White, and C. D. Schuman, "The Cart-Pole Application as a Benchmark for Neuromorphic Computing," *Journal of Low Power Electronics and Applications*, vol. 15, no. 1, p. 5, 2025.

[12] T. Mastin, N. Anderson, S. McNeal, M. Tubbin, and C. Teuscher, "Fractional-order systems for neuromorphic computing: Software and hardware opportunities and challenges," *npj Unconventional Computing*, vol. 3, no. 1, p. 24, 2026.

[13] R. Kannan, M. Gulhane, K. Mirza, S. Maurya, S. Kumar, and N. Rakesh, "Architectural Innovations for AI Chips, Testing, and High Performance Computing," in *2024 IEEE 6th International Conference on Cybernetics, Cognition and Machine Learning Applications (ICCCMLA)*, 2024, pp. 365–369.

[14] K. Yamazaki, V.-K. Vo-Ho, D. Bulsara, and N. Le, "Spiking Neural Networks and Their Applications: A Review," *Brain Sciences*, vol. 12, no. 7, p. 863, 2022.

[15] J. Tang, F. Yuan, X. Shen, Z. Wang, M. Rao, Y. He, Y. Sun, X. Li, W. Zhang, Y. Li, B. Gao, H. Qian, G. Bi, S. Song, J. J. Yang, and H. Wu, "Bridging Biological and Artificial Neural Networks with Emerging Neuromorphic Devices: Fundamentals, Progress, and Challenges," *Advanced Materials*, vol. 31, no. 49, p. 1902761, 2019.

[16] W. Teka, R. K. Upadhyay, and A. Mondal, "Fractional-order leaky integrate-and-fire model with long-term memory and power law dynamics," *Neural Networks*, vol. 93, pp. 110–125, 2017.

[17] P. Vazquez-Guerrero, R. Tuladhar, C. Psychalinos, A. Elwakil, M. J. Chacron, and F. Santamaria, "Fractional order memcapacitive neuromorphic elements reproduce and predict neuronal function," *Scientific Reports*, vol. 14, no. 1, p. 5817, 2024.

[18] A. Ali, K. Bingi, R. Ibrahim, P. A. M. Devan, and K. B. Devika, "A review on FPGA implementation of fractional-order systems and PID controllers," *AEU - International Journal of Electronics and Communications*, vol. 177, p. 155218, 2024.

[19] D. Thomas and W. Luk, "FPGA Accelerated Simulation of Biologically Plausible Spiking Neural Networks," in *2009 17th IEEE Symposium on Field Programmable Custom Computing Machines*, 2009, pp. 45–52.

[20] F. Shama, S. Haghiri, and M. A. Imani, "FPGA Realization of Hodgkin-Huxley Neuronal Model," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 28, no. 5, pp. 1059–1068, 2020.

[21] G. Ghanbarpour, M. A. Chaudhary, M. Assaad, and M. Ghanbarpour, "Hardware-efficient implementation of FitzHugh–Nagumo neural networks on FPGA: A scalable approach for neuromorphic system design," *AEU - International Journal of Electronics and Communications*, vol. 200, p. 155886, 2025.

[22] M. Heidarpour, A. Ahmadi, and R. Rashidzadeh, "A CORDIC Based Digital Hardware For Adaptive Exponential Integrate and Fire Neuron," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 63, no. 11, pp. 1986–1996, 2016.

[23] S. A. Malik and A. H. Mir, "FPGA Realization of Fractional Order Neuron," *Applied Mathematical Modelling*, vol. 81, pp. 372–385, 2020.

[24] M. F. Tolba, A. H. Elsafty, M. Armanyos, L. A. Said, A. H. Madian, and A. G. Radwan, "Synchronization and FPGA realization of fractional-order Izhikevich neuron model," *Microelectronics Journal*, vol. 89, pp. 56–69, 2019.

[25] A. Gautam, P. Date, S. Kulkarni, R. Patton, and T. Potok, "NeuroCoreX: An Open-Source FPGA-Based Spiking Neural Network Emulator with On-Chip Learning," 2025, arXiv:2506.14138. [Online]. Available: https://arxiv.org/abs/2506.14138

[26] Y. Liu, Y. Chen, W. Ye, and Y. Gui, "FPGA-NHAP: A General FPGA-Based Neuromorphic Hardware Acceleration Platform With High Speed and Low Power," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 6, pp. 2553–2566, 2022.

[27] J. P. Mitchell, C. D. Schuman, R. M. Patton, and T. E. Potok, "Caspian: A Neuromorphic Development Platform," in *Proceedings of the 2020 Annual Neuro-Inspired Computational Elements Workshop*, ser. NICE '20. New York, NY, USA: Association for Computing Machinery, 2020, pp. 1–6.

[28] A. Carpegna, A. Savino, and S. D. Carlo, "Spiker+: A Framework for the Generation of Efficient Spiking Neural Networks FPGA Accelerators for Inference at the Edge," *IEEE Transactions on Emerging Topics in Computing*, vol. 13, no. 3, pp. 784–798, 2025.

[29] J. Li, G. Shen, D. Zhao, Q. Zhang, and Y. Zeng, "FireFly v2: Advancing Hardware Support for High-Performance Spiking Neural Network With a Spatiotemporal FPGA Accelerator," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 9, pp. 2647–2660, 2024.

[30] A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-13, no. 5, pp. 834–846, 1983.

[31] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. D. Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, "Gymnasium: A Standard Interface for Reinforcement Learning Environments," Nov. 2025.

[32] L. Zanatta, F. Barchi, S. Manoni, S. Tolu, A. Bartolini, and A. Acquaviva, "Exploring spiking neural networks for deep reinforcement learning in robotic tasks," *Scientific Reports*, vol. 14, no. 1, p. 30648, 2024.

[33] D. Haşegan, M. Deible, C. Earl, D. D'Onofrio, H. Hazan, H. Anwar, and S. A. Neymotin, "Training spiking neuronal networks to perform motor control using reinforcement and evolutionary learning," *Frontiers in Computational Neuroscience*, vol. 16, p. 1017284, 2022.

[34] S. Kumar, "Balancing a CartPole System with Reinforcement Learning – A Tutorial," 2020, arXiv:2006.04938. [Online]. Available: https://arxiv.org/abs/2006.04938

[35] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[36] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *ICLR (Poster)*, San Juan, Puerto Rico, 2016. [Online]. Available: http://arxiv.org/abs/1509.02971

[37] A. Paszke and M. Towers, "Reinforcement Learning (DQN) Tutorial — PyTorch Tutorials 2.9.0+cu128 documentation," [Online] Available: https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html, [Accessed: Oct. 10, 2025].

[38] J. K. Eshraghian, M. Ward, E. O. Neftci, X. Wang, G. Lenz, G. Dwivedi, M. Bennamoun, D. S. Jeong, and W. D. Lu, "Training Spiking Neural Networks Using Lessons From Deep Learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.

[39] "OSS CAD Suite," [GitHub repository] Available: https://github.com/YosysHQ/oss-cad-suite-build, 2025, [Accessed: Jul. 15, 2025]. [Online]. Available: https://github.com/YosysHQ/oss-cad-suite-build

[40] "Vivado design suite user guide," Advanced Micro Devices, Inc., 2025, version 2025.1. [Online]. Available: https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html

[41] "Optuna: An open source hyperparameter optimization framework," [Online] Available: https://optuna.org/, 2026, [Accessed: Feb. 10, 2026].

[42] G. Liu, W. Deng, X. Xie, L. Huang, and H. Tang, "Human-Level Control Through Directly Trained Deep Spiking Q-Networks," *IEEE Transactions on Cybernetics*, vol. 53, no. 11, pp. 7187–7198, 2023.

[43] Y. Sun, Y. Zeng, and Y. Li, "Solving the spike feature information vanishing problem in spiking deep Q network with potential based normalization," *Frontiers in Neuroscience*, vol. 16, 2022.

[44] W. Tan, D. Patel, and R. Kozma, "Strategy and Benchmark for Converting Deep Q-Networks to Event-Driven Spiking Neural Networks," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 11, pp. 9816–9824, 2021.

[45] J. Eshraghian, "snnTorch Documentation," [Online] Available: https://snntorch.readthedocs.io/en/latest/, 2023, [Accessed: Jul. 15, 2025].

[46] M. C. Machado, M. G. Bellemare, E. Talvitie, J. Veness, M. Hausknecht, and M. Bowling, "Revisiting the Arcade Learning Environment: Evaluation Protocols and Open Problems for General Agents," *Journal of Artificial Intelligence Research*, vol. 61, pp. 523–562, 2018.

[47] "Gymnasium," [GitHub repository] Available: https://github.com/Farama-Foundation/Gymnasium, 2025, [Accessed: Oct. 10, 2025].

[48] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems*, vol. 34. Curran Associates, Inc., 2021, pp. 29 304–29 320.

[49] M. McCloskey and N. J. Cohen, "Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem," *Psychology of Learning and Motivation*, vol. 24, pp. 109–165, 1989.

[50] C. Colas, O. Sigaud, and P.-Y. Oudeyer, "How Many Random Seeds? Statistical Power Analysis in Deep Reinforcement Learning Experiments," 2018, arXiv:1806.08295. [Online]. Available: https://arxiv.org/abs/1806.08295

[51] A. H. Ali, M. Navardi, and T. Mohsenin, "Energy-Aware FPGA Implementation of Spiking Neural Network with LIF Neurons," in *Proceedings of the TinyML Research Symposium 2024*, 2024.

[52] S. M. Mishra, H. S. Shekhawat, J. Pidanic, and G. Trivedi, "FPGA-Based Adaptive LIF Neuron for High-Speed, Energy-Efficient Spiking Neural Network," *IEEE Transactions on Circuits and Systems for Artificial Intelligence*, vol. 2, no. 3, pp. 276–285, 2025.

[53] J. Zhang, Y. Wang, Y. Cai, B. Bi, Q. Chen, and Y. Zhang, "A Low Power Spiking Neural Network Accelerator on FPGA for Real-Time Edge Computing," in *2025 Joint International Conference on Automation-Intelligence-Safety (ICAIS) & International Symposium on Autonomous Systems (ISAS)*, 2025, pp. 1–6.

[54] C. Ge, Y. Peng, Z. Li, Q. Kang, X. Fu, X. Li, Q. Zhang, J. Ren, and Z.-J. Zha, "Fractional-order Spiking Neural Network," in *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. [Online]. Available: https://openreview.net/forum?id=NJhBSLJ0nL